MACHINE LEARNING / MODULE 2

Ensembles, interpretation, and pipelines

Voting, bagging, boosting, honest explanations, and a predictor you can deploy

Full day · 09:00–17:00 | 6 teaching parts | 2 laboratories | Builds on Module 1

Today's plan

09:00	Welcome, warm-up, and a drilling case	Lecture
09:15	Part 1: Why combining models works	Lecture
09:50	Part 2: Bagging and random forests	Lecture
10:35	Break	Break
10:50	Part 3: From AdaBoost to gradient boosting	Lecture
11:45	Lab 1: tree, forest, and boosting compared	Lab
12:45	Lunch	Lunch
13:45	Part 4: Boosting libraries and stacking	Lecture
14:15	Part 5: Interpreting ensemble models	Lecture
15:00	Break	Break
15:15	Part 6: Pipelines, leakage, and deployment	Lecture
15:50	Lab 2: importance, pipelines, and tuning	Lab
16:45	Evidence review and exit check	Discussion

What you will be able to do

By the end of the day you will be able to:

1. Explain when voting or averaging reduces error, using member accuracy and correlation.
2. Fit and tune random forests, and weigh out-of-bag scores against grouped validation.
3. Trace AdaBoost and gradient-boosting updates and control them with depth, learning rate, and early stopping.
4. Choose between a forest, boosting, and stacking under a validation and deployment budget.
5. Interpret an ensemble with permutation importance, PDP/ICE, and Shapley values, and state their limits.
6. Package preprocessing and model in one pipeline, avoid leakage, and save it with its context.

PRACTICE

4 min · Pairs

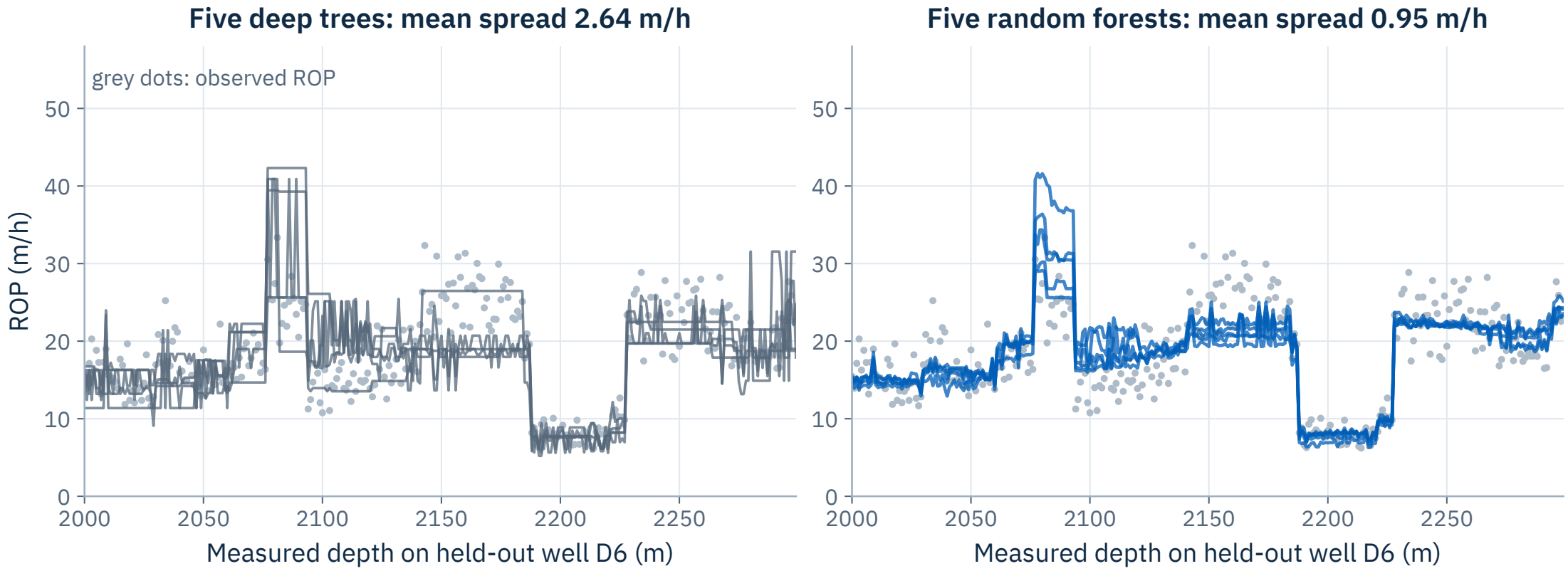
A fragile model can become a useful component

Three classifiers produce event probabilities 0.2, 0.7, and 0.6.

1. What is their mean probability?
2. What prediction results from a 0.5 threshold?
3. What if all three learned the same systematic error?

Retrained trees disagree; retrained forests agree

Five resamples of wells D1–D5; every model predicts rate of penetration (ROP) on well D6.



Synthetic drilling data (seed 11). Spread = standard deviation across the five models, averaged over depth: trees 2.64 m/h, forests 0.95 m/h.

PART 1 OF 6

01

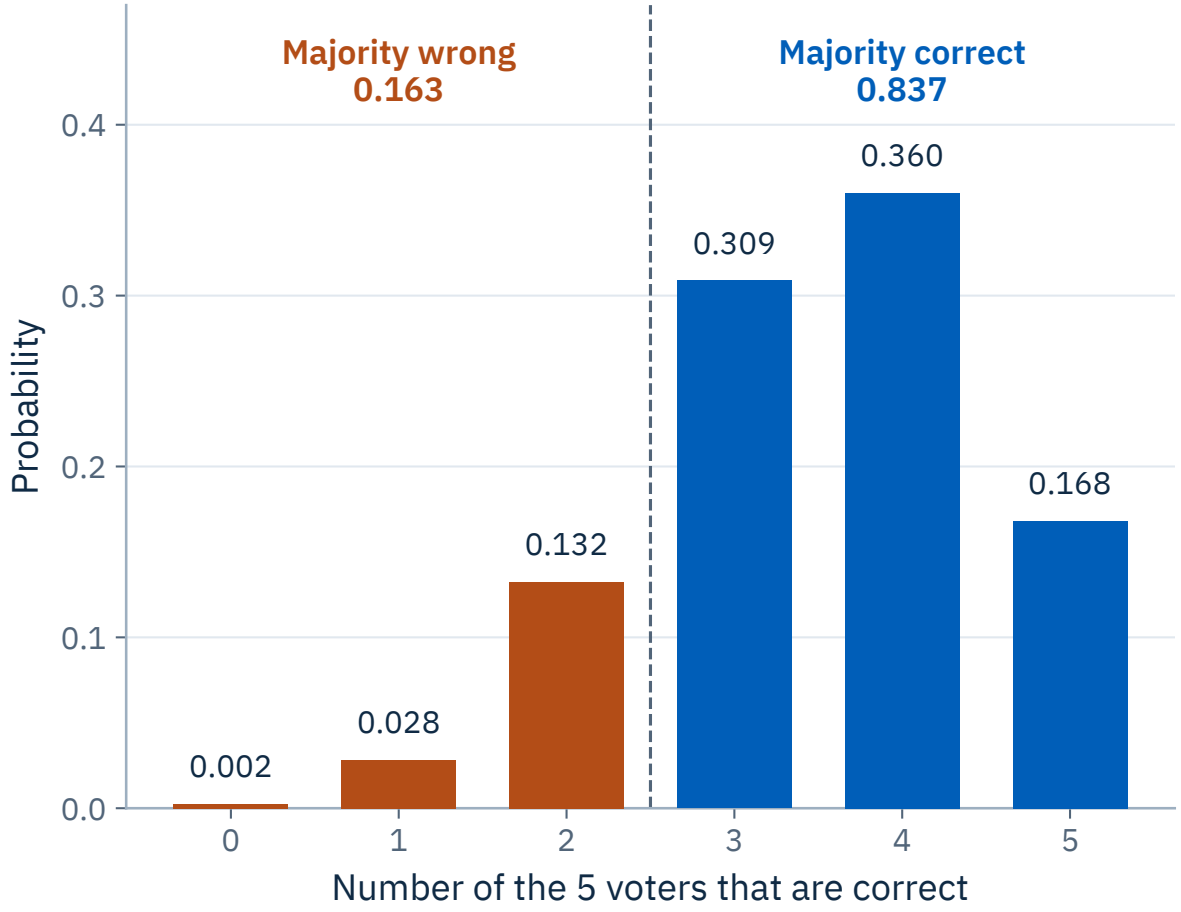
Why combining models works

Many imperfect models can beat one, if their errors differ.

- Majority votes and the binomial argument
- Correlation and the variance floor
- Hard and soft voting
- Diversity by design

Independent errors rarely line up

- Five voters, each right 70% of the time, with independent errors.
- The vote fails only when three or more are wrong: probability 0.163.
- Majority accuracy is 0.837, better than any single member.

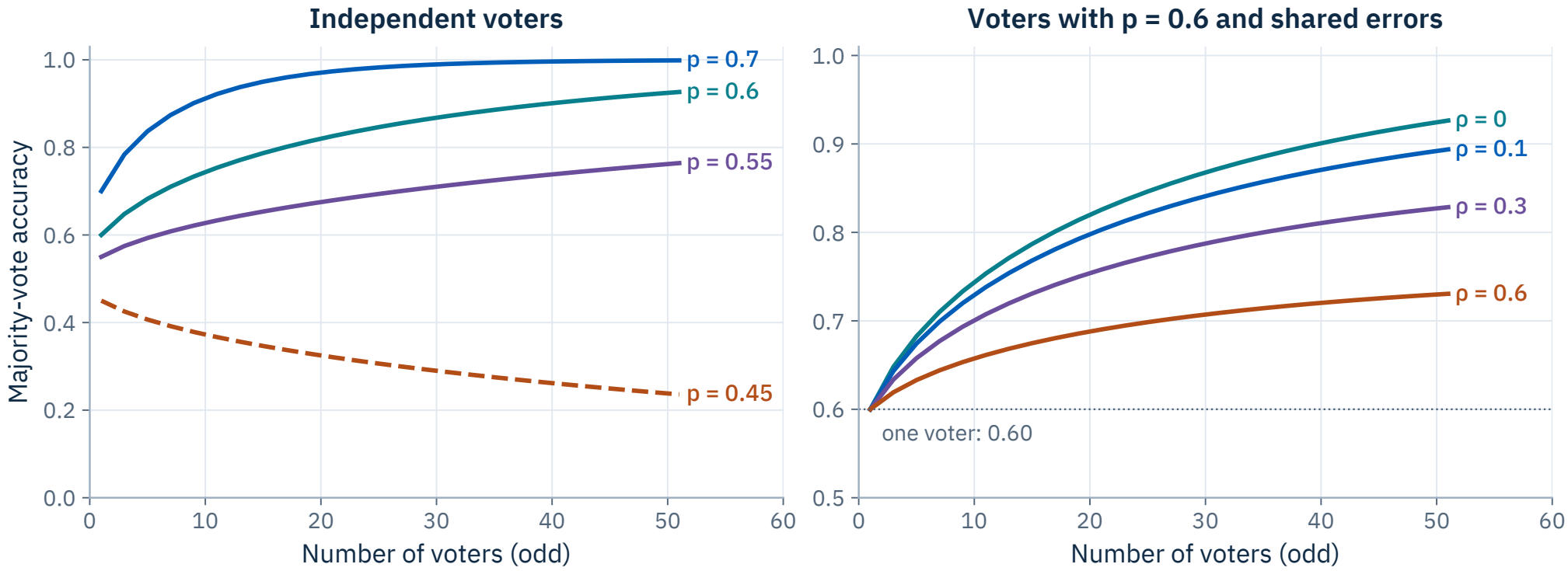


Exact binomial probabilities for n = 5 voters, each correct with p = 0.7.

◆ Independence is the assumption doing the work.

Voting helps only accurate, independent members

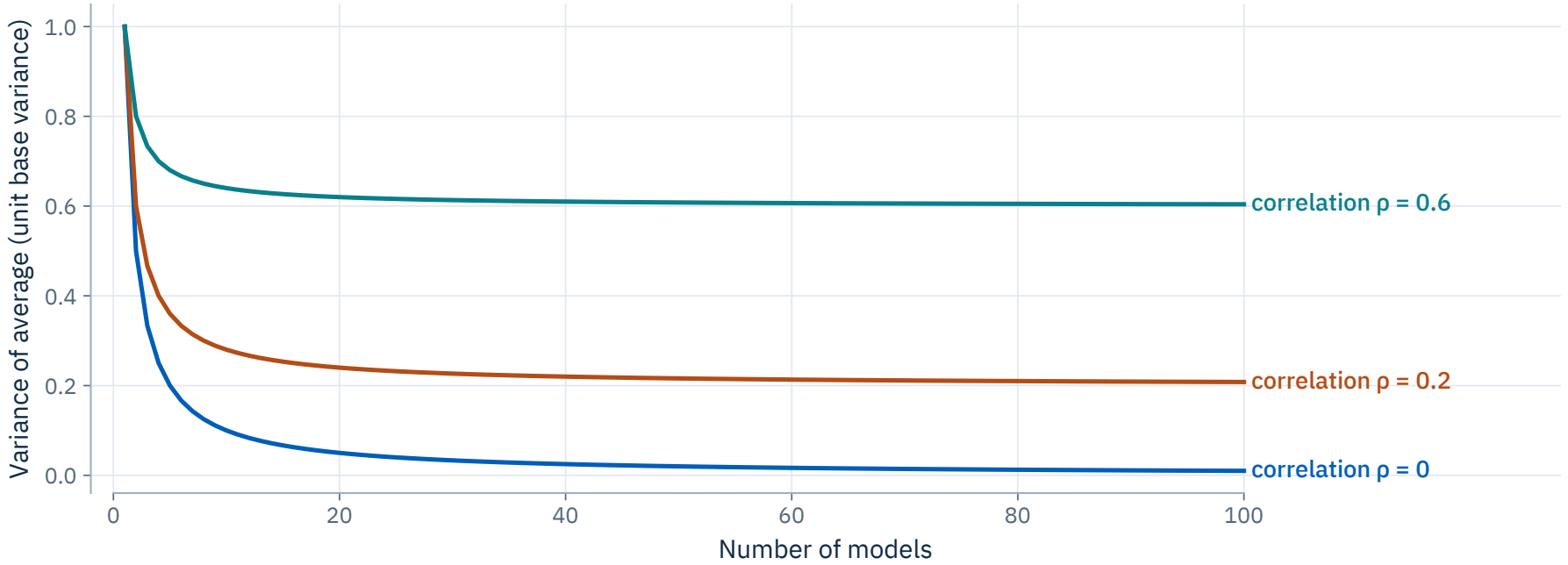
Left: members above 50% improve with numbers, members below 50% get worse. Right: shared errors cap the gain.



Exact binomial majority probabilities. Right: with probability ρ all voters copy one shared vote; otherwise they vote independently.

Correlation leaves a variance floor

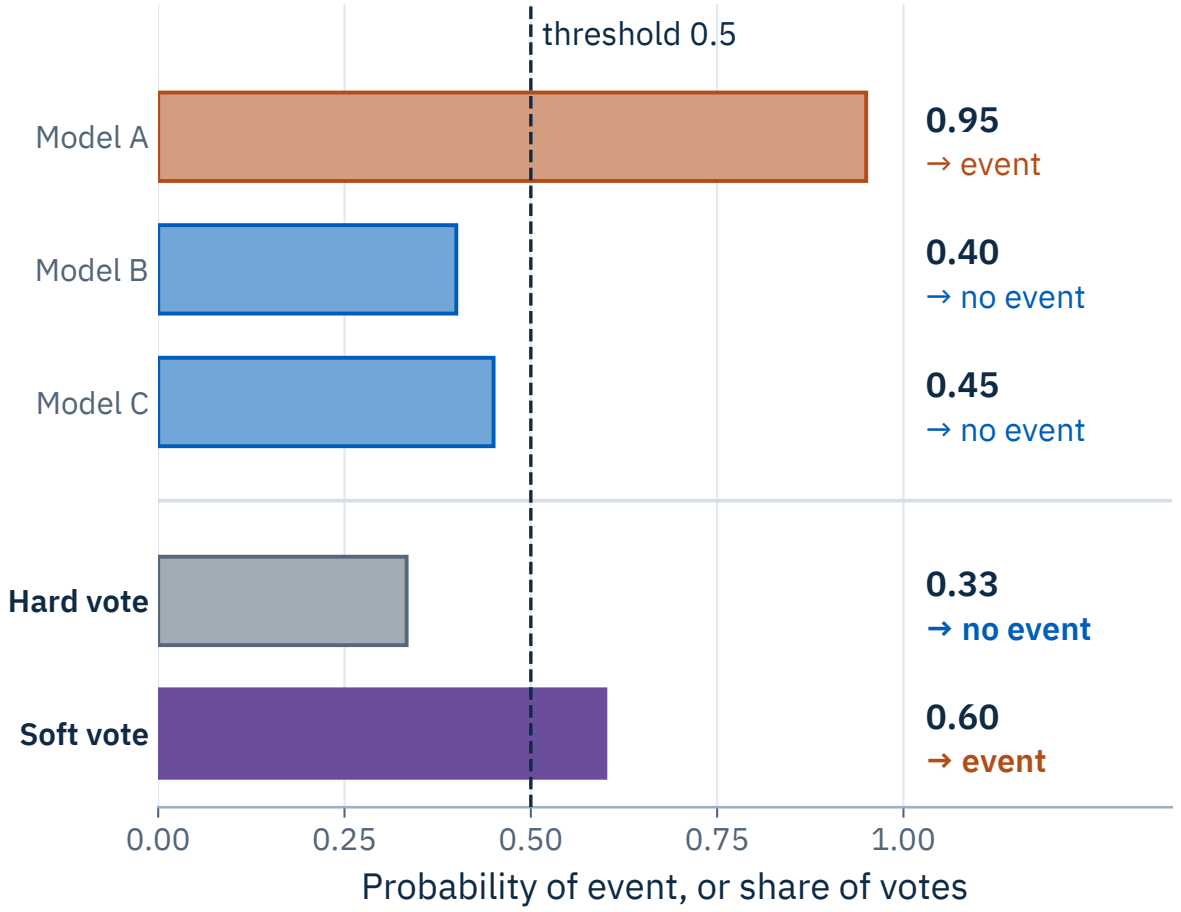
$$\mathrm{Var}(\bar{f}) = \sigma^2\left(\rho + \frac{1-\rho}{M}\right)$$



Analytical curves for unit model variance and several assumed error correlations; not a measured benchmark. Example: ρ = 0.2, M = 10 gives 0.28; the floor is 0.20.

Hard and soft voting can disagree

- Hard vote: each model casts one vote for its predicted class.
- Soft vote: average the predicted probabilities, then apply the threshold.
- Soft voting uses confidence, so members need sensible probabilities.



Exact arithmetic for three illustrative probabilities.

◆ Same three models, opposite decisions.

VotingClassifier mixes model families

```
from sklearn.ensemble import VotingClassifier
members = [
    ("logit", make_pipeline(StandardScaler(), LogisticRegression())),
    ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(15))),
    ("forest", RandomForestClassifier(300, random_state=0)),
]
vote = VotingClassifier(members, voting="soft")

cv = GroupKFold(n_splits=4)                    # new wells in every fold
for name, model in members + [("soft vote", vote)]:
    s = cross_val_score(model, X, y, groups=wells, cv=cv)
    print(f"{name:10s} {s.mean():.3f}")
```

OUTPUT

```
logit      0.857
knn        0.813
forest     0.776
soft vote  0.841
```

- The vote trails its best member here: combining is a hypothesis to test.

Diversity comes from rows, features, and algorithms

Different rows

- Bootstrap samples (bagging, random forests)
- Row subsampling in stochastic boosting

Different features

- Random subset of features at each split
- Random feature subsets per model

Different algorithms

- Linear, neighbours, and trees in one vote
- A learned combination (stacking)

Different randomization

- Random split thresholds (extra-trees)
- Different seeds and initializations

◆ Diversity helps only while each member stays better than chance.

PRACTICE

5 min · Pairs

How much can three voters gain?

Assume independent errors unless stated otherwise.

1. Three classifiers are each correct 80% of the time. What is the majority-vote accuracy?
2. Repeat for three classifiers that are each correct 40% of the time.
3. What is the accuracy if the three are identical copies of one 80% model?

PART 2 OF 6

02

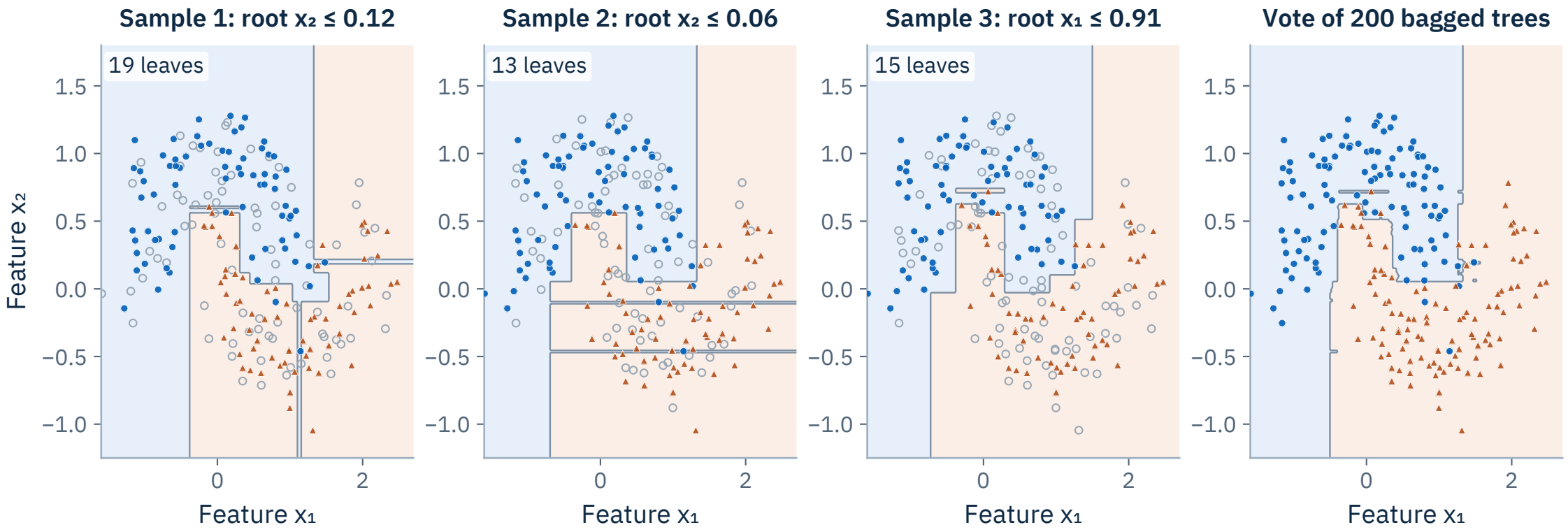
Bagging and random forests

Average many deep trees trained on perturbed data.

- Why single trees are unstable
- Bootstrap aggregation and feature subsampling
- Out-of-bag estimates and their limits
- A facies case on a new well

A deep tree changes with every bootstrap sample

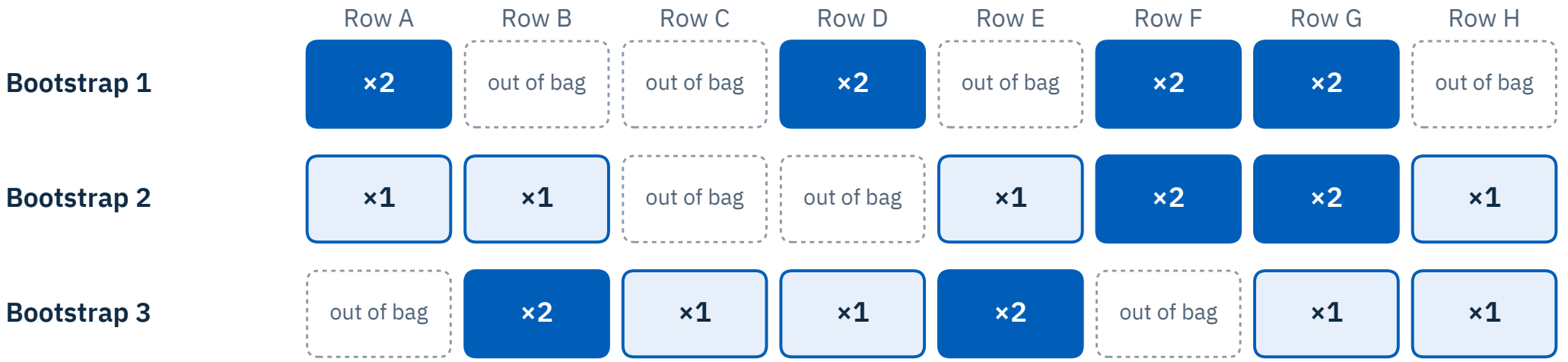
Same 220 rows, three bootstrap resamples: different root splits, leaf counts, and boundaries.



Synthetic two-moons data (seed 42); bootstrap seed 18; unpruned scikit-learn trees. Hollow markers: rows left out of that sample.

Bootstrap samples contain duplicates

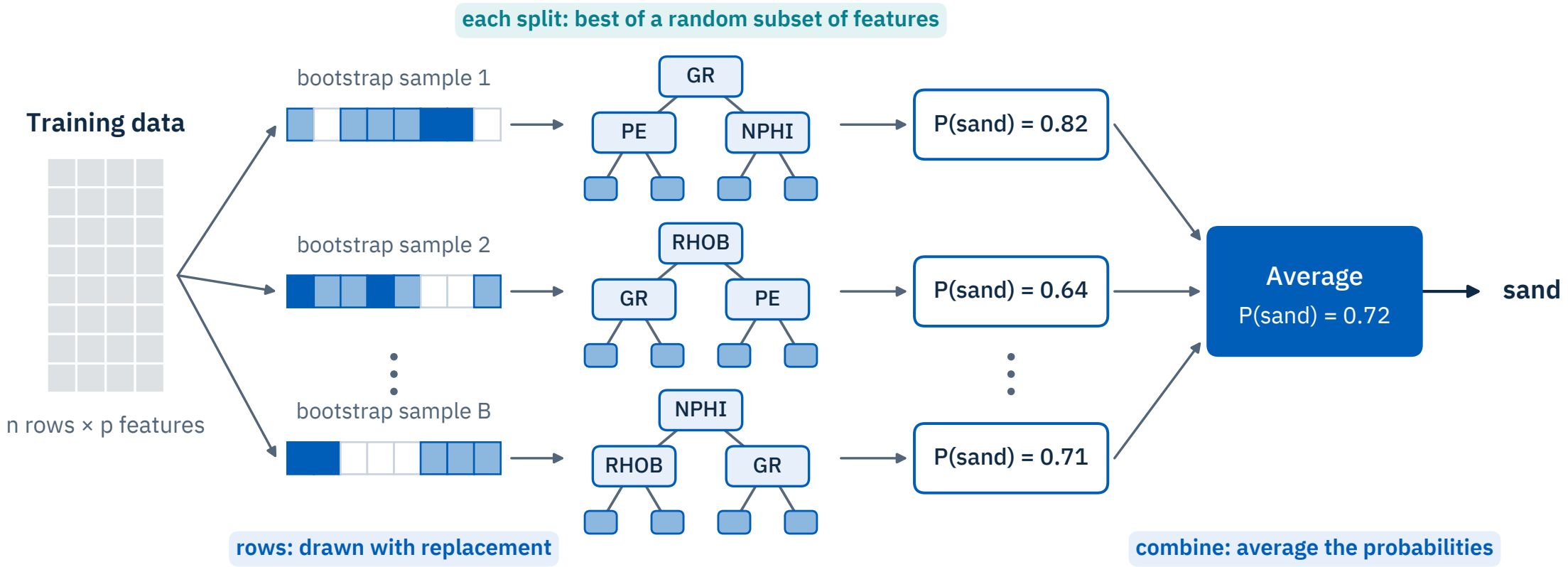
A row drawn twice counts twice in that tree's fit; rows never drawn are its out-of-bag rows.



Each bootstrap draws 8 rows with replacement: some rows repeat, others are left out (out of bag).

One small synthetic bootstrap example (seed 42), n = 8 rows.

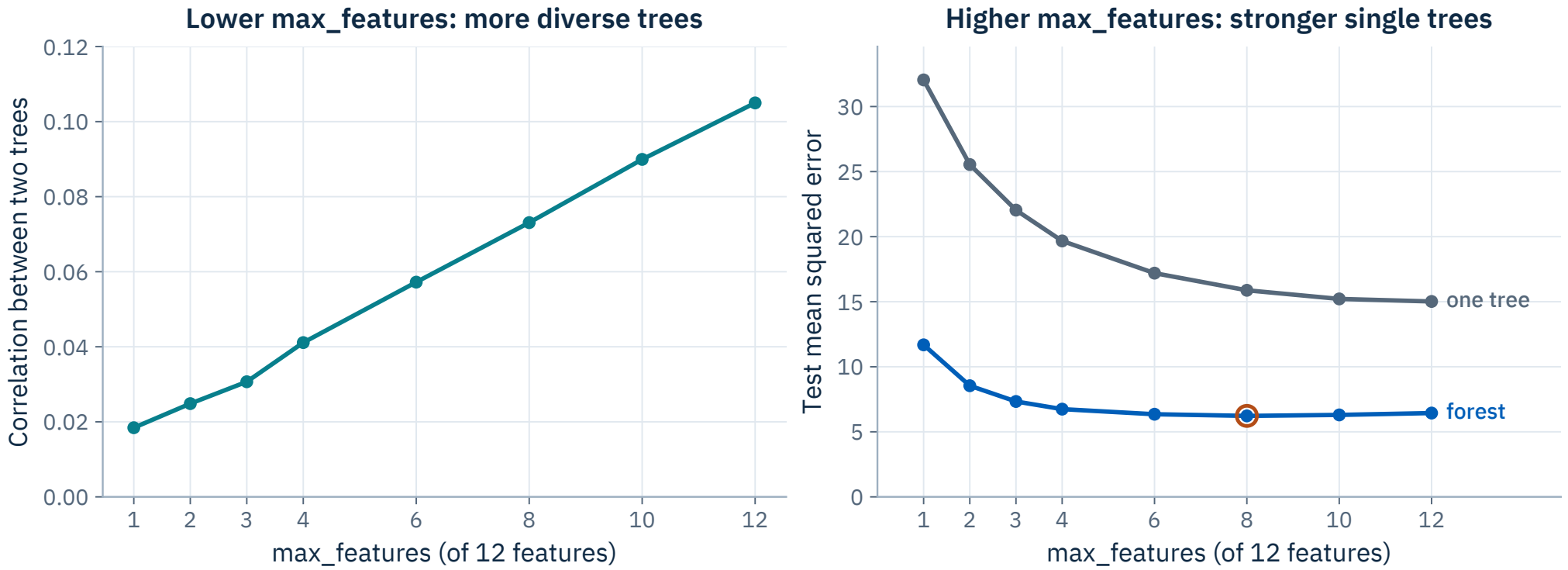
A random forest decorrelates trees before voting



Schematic. Feature names and probabilities are illustrative; the average of 0.82, 0.64, and 0.71 is 0.72.

Feature subsampling trades strength for diversity

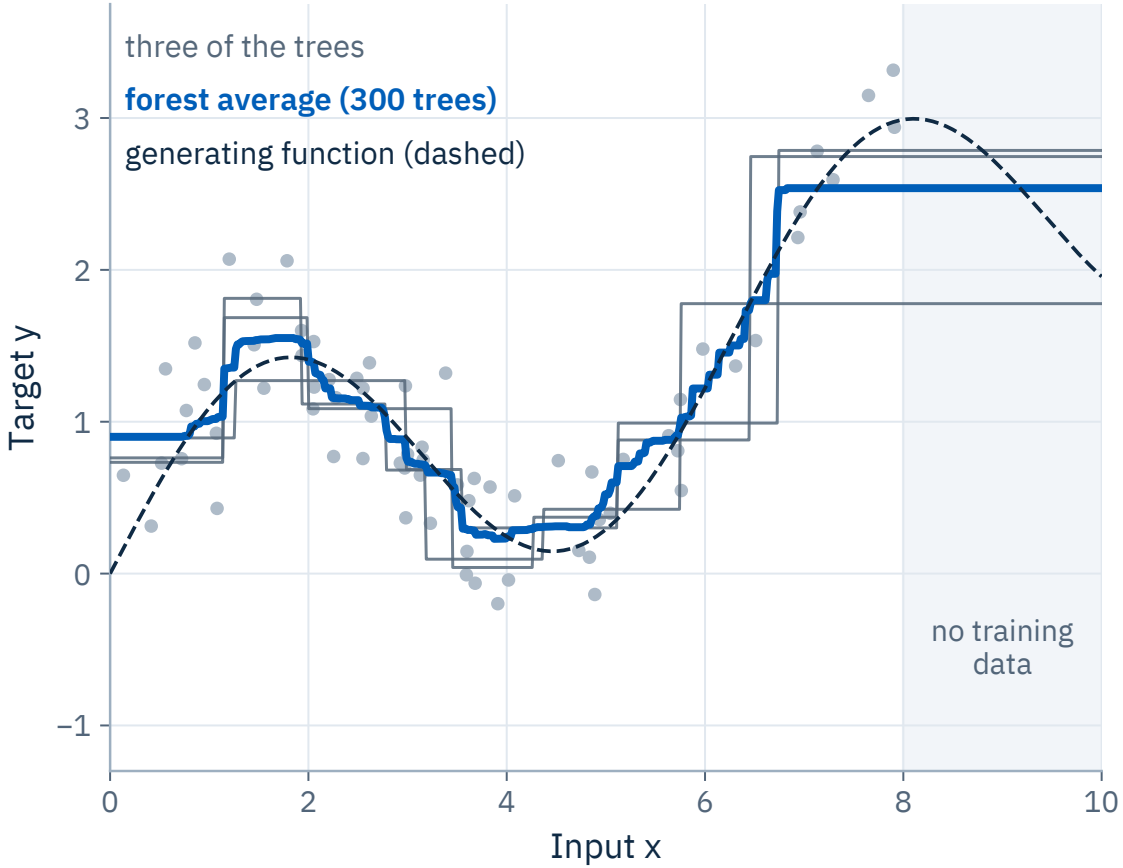
Fewer features per split: less correlated trees but weaker ones. The forest does best in between.



Friedman #1 synthetic regression (12 features, 5 informative): 16 independent training sets of 300 rows, forests of 60 trees. Circle: lowest forest error.

Averaging turns jagged trees into a smoother fit

- Each tree is a step function fitted to its own bootstrap sample.
- The forest averages 300 step functions: steps blur and noise cancels.
- Beyond the data (shaded) the average stays flat: trees cannot extrapolate.



Synthetic data y = sin(x) + 0.25x + noise, n = 70 on 0 ≤ x ≤ 8 (seed 8); 300 trees, min_samples_leaf = 5.

◆ Forests average predictions; classifiers average class probabilities.

Forest controls have practical meanings

Parameter	What it changes	Diagnostic
n_estimators	Number of trees	Score stability and runtime
max_features	Features considered at each split	Tree diversity and fit
min_samples_leaf	Smallest terminal group	Smoothing and rare patterns
max_depth	Longest decision path	Complexity and memory

Tune leaf size and feature subsampling before expanding the search.

◆ Defaults are starting points: max_features is "sqrt" for classifiers and 1.0 for regressors.

Each row is scored only by trees that never saw it

- About a third of the trees leave out any given row.
- A row's out-of-bag prediction uses only those trees.
- The OOB score needs no separate validation fold.

Trees (each fitted to one bootstrap sample)

	T1	T2	T3	T4	T5	T6	OOB vote
W1 1500.0 m	×1	×1	out	×1	out	out	T3, T5, T6
W1 1500.5 m	×1	×1	×1	×1	out	×2	T5
W1 1501.0 m	×2	×2	×1	×1	out	×1	T5
W1 1501.5 m	×1	out	×1	×2	×3	out	T2, T6
W1 1502.0 m	×2	out	×2	out	out	×2	T2, T4, T5
W1 1502.5 m	out	×1	×1	out	×2	×1	T1, T4
W1 1503.0 m	×1	×1	×2	×2	×1	×1	none
W1 1503.5 m	out	×1	×1	out	×1	×1	T1, T4
W1 1504.0 m	×2	×2	out	×1	×2	out	T3, T6
W1 1504.5 m	out	×1	×1	×2	×1	×2	T1

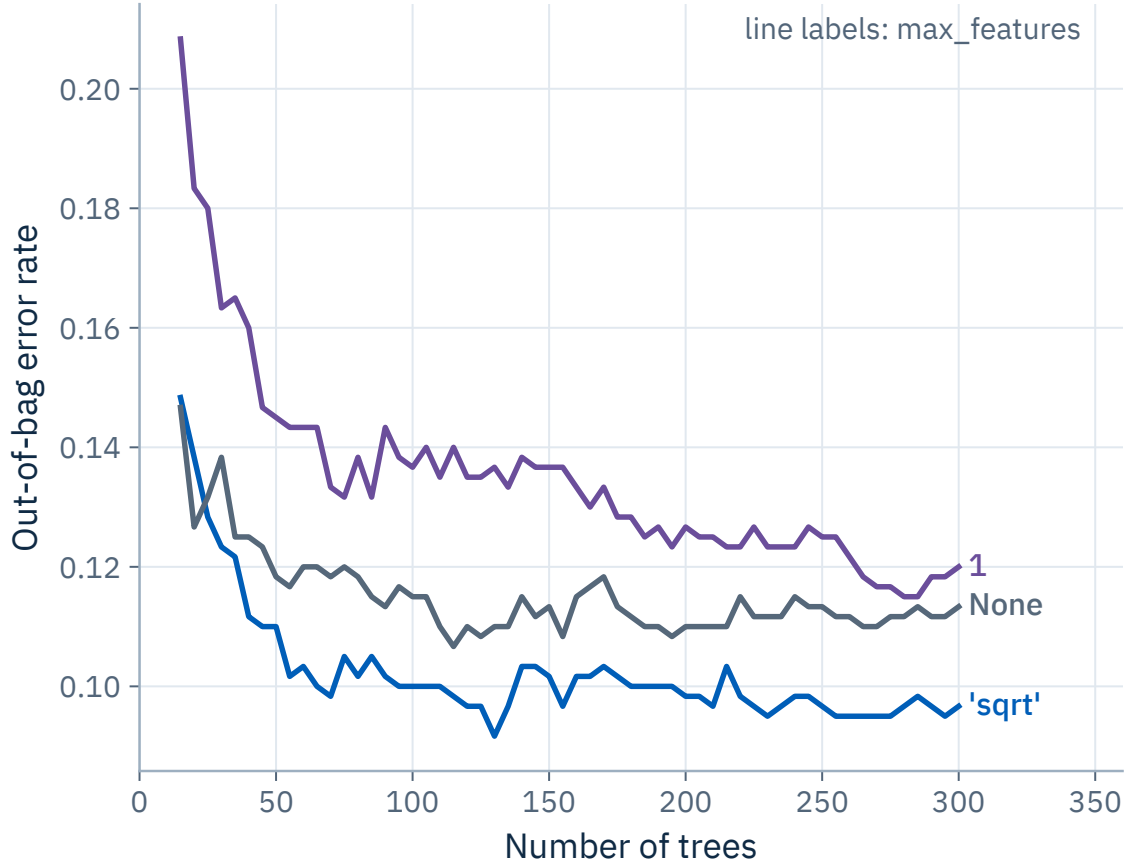
17 of 60 cells out of bag (28%); expected $(1 - 1/n)^n = 35\%$

◆ Neighbouring depth rows are often in-bag when a row is out of bag.

Bootstrap draws for 10 adjacent depth rows of one synthetic well and 6 trees (seed 2).

Out-of-bag error settles as trees are added

- Error falls fast, then flattens: extra trees stabilize the average.
- Pick n_estimators on the flat part, for cost rather than accuracy.
- max_features shifts the level of the curve, not its shape.



Synthetic classification data (600 rows, 10 features, 5% label noise, seed 3); forests grown with warm_start, seed 0.

A forest reports its out-of-bag score for free

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupKFold, cross_val_score

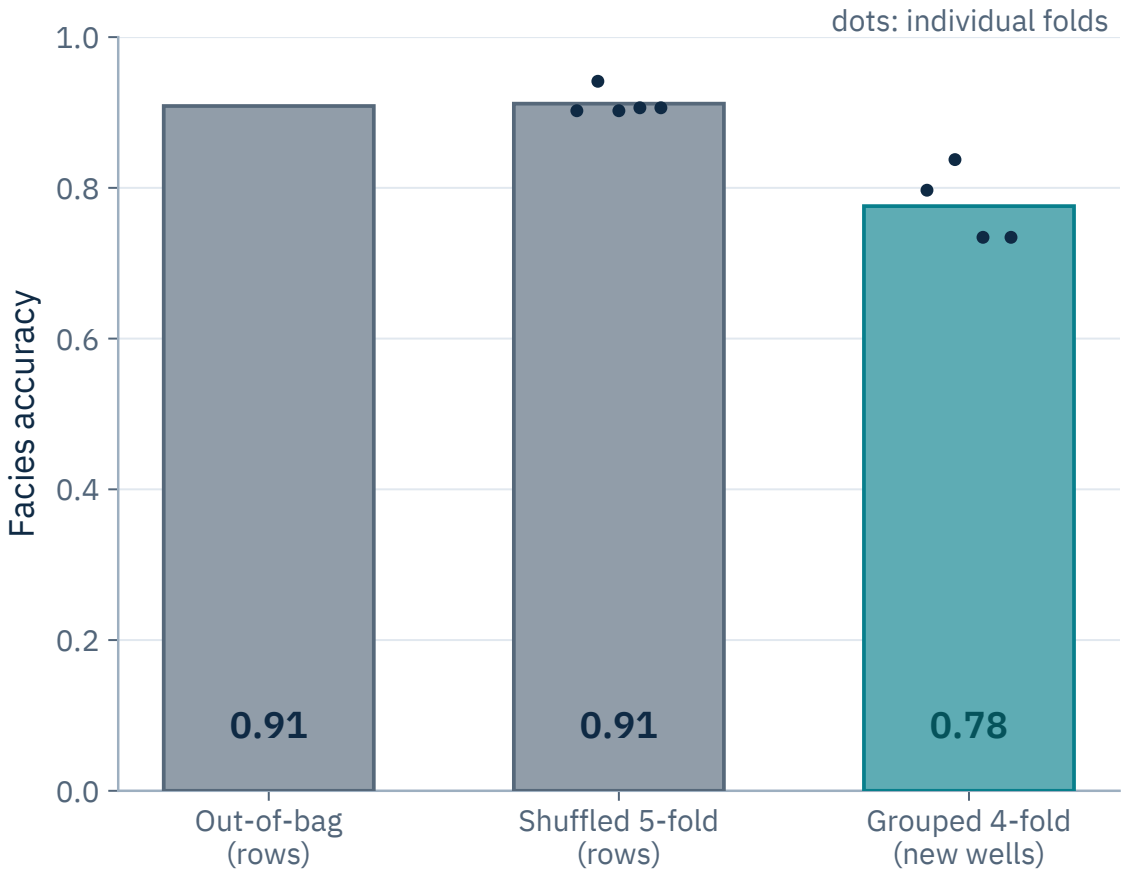
forest = RandomForestClassifier(
    n_estimators=300,        # enough for a stable average
    max_features="sqrt",     # features tried at each split
    min_samples_leaf=3,      # smallest leaf allowed
    oob_score=True, n_jobs=-1, random_state=0,
)
forest.fit(X, y)
print(f"OOB accuracy     {forest.oob_score_:.3f}")
s = cross_val_score(forest, X, y, groups=wells, cv=GroupKFold(4))
print(f"grouped accuracy {s.mean():.3f}")
```

OUTPUT

```
OOB accuracy     0.911
grouped accuracy 0.785
```

Out-of-bag evaluation has assumptions

- Each training row receives predictions from trees that did not sample it.
- This is an internal estimate for approximately independent rows.
- It does not protect against leakage across related well samples.
- Keep grouped or chronological evaluation for those deployment questions.



Synthetic facies logs, 8 wells with well offsets and depth-correlated noise (seed 7); forest of 300 trees, seed 0.

Extra-trees also randomize the split thresholds

Random forest

- Bootstrap sample of rows for each tree
- Best threshold for each candidate feature
- Out-of-bag score possible (bootstrap=True)

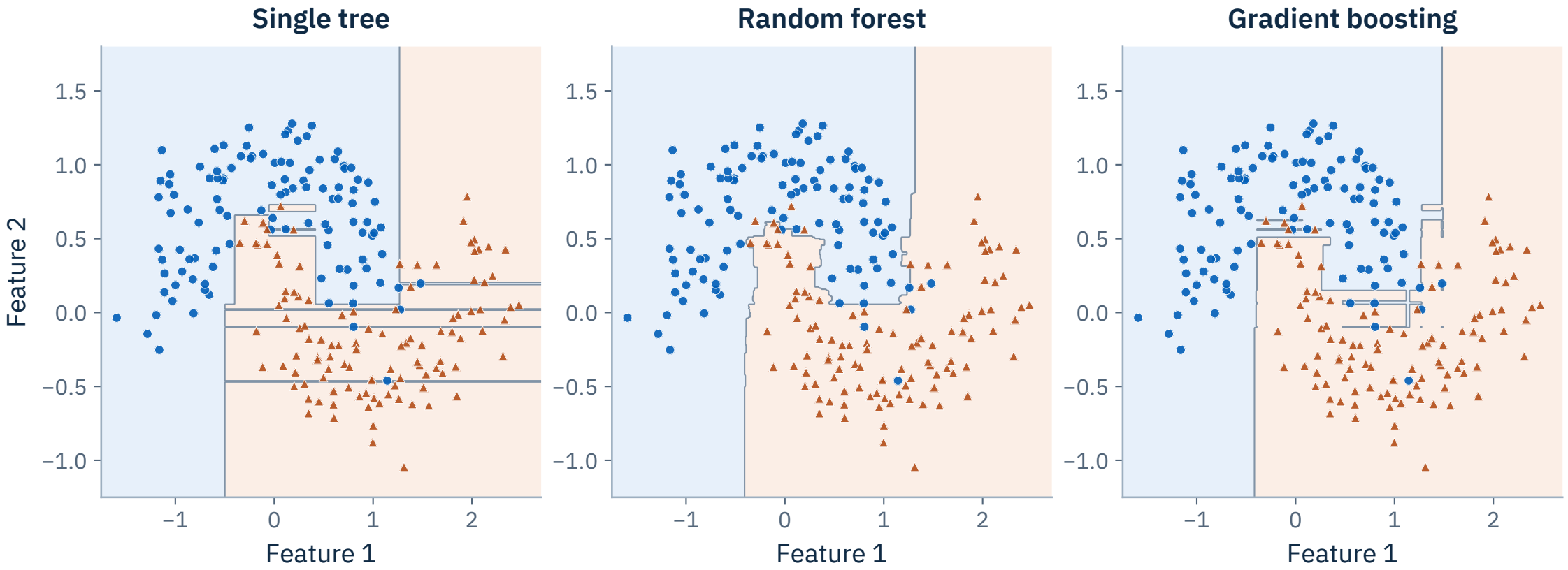
vs

Extra-trees

- All training rows by default (bootstrap=False)
- One random threshold per candidate feature
- Faster to train; often smoother boundaries

◆ Try both with the same folds; neither wins everywhere.

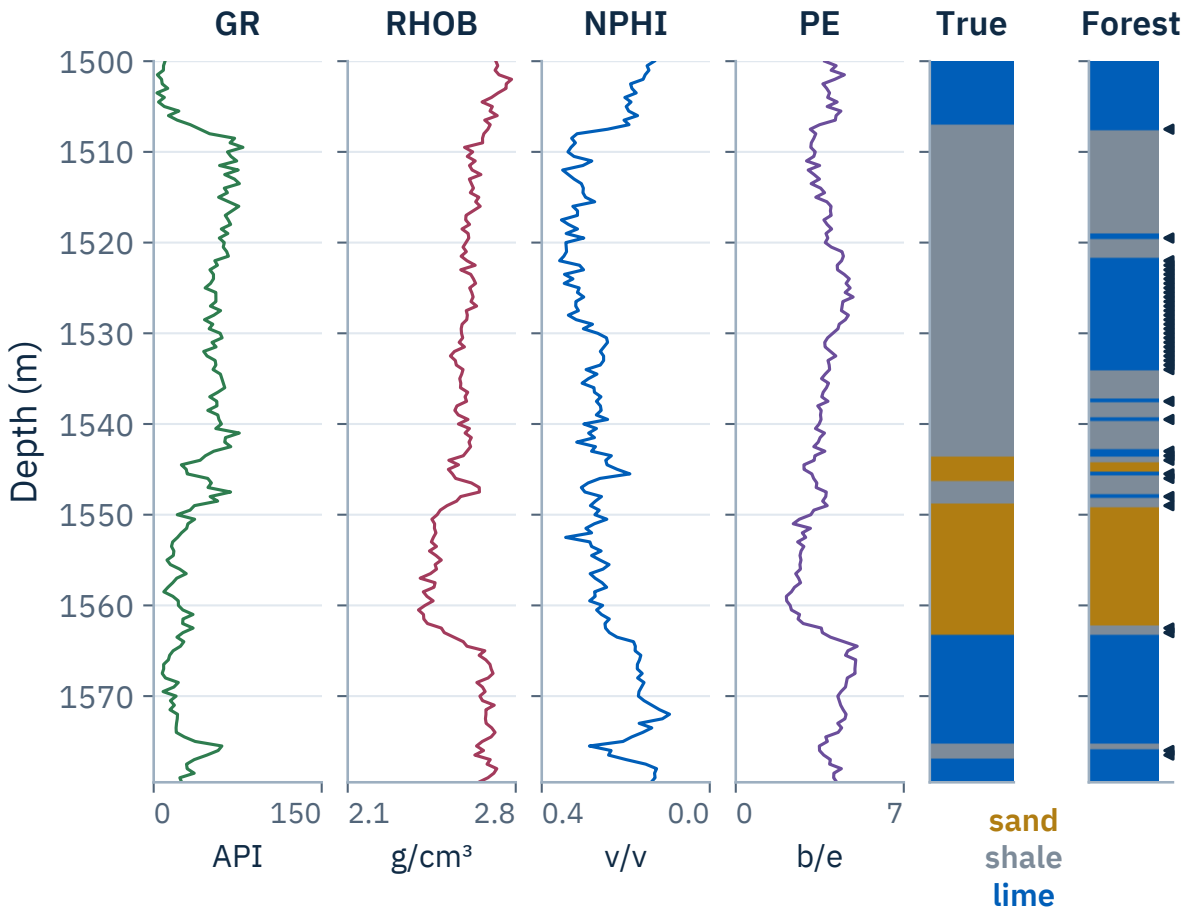
Ensembles change decision boundaries



Tree, forest, and boosting fits on the same synthetic data. Boundary appearance alone does not establish generalization quality.

A forest classifies facies on a held-out well

- Trained on wells W1–W7, scored on W8 only.
- 75% of W8's rows are correct.
- One 12 m shale interval is called limestone; most other errors sit at bed boundaries.



Synthetic logs; forest of 300 trees (min_samples_leaf 3, seed 0).

◆ Report errors per well and per facies, not only overall.

PRACTICE

5 min · Pairs

Which forest change would you try first?

Scenario: a forest predicts ESP trips from sensor summaries of 12 wells. Training accuracy 1.00, OOB 0.93, grouped CV by well 0.79.

1. Grow 2,000 trees instead of 300: what changes?
2. Raise min_samples_leaf from 1 to 20: what might improve, and how do you check?
3. Switch max_features from "sqrt" to all features: likely effect?
4. Add the well identifier as a feature: acceptable?

PART 3 OF 6

03

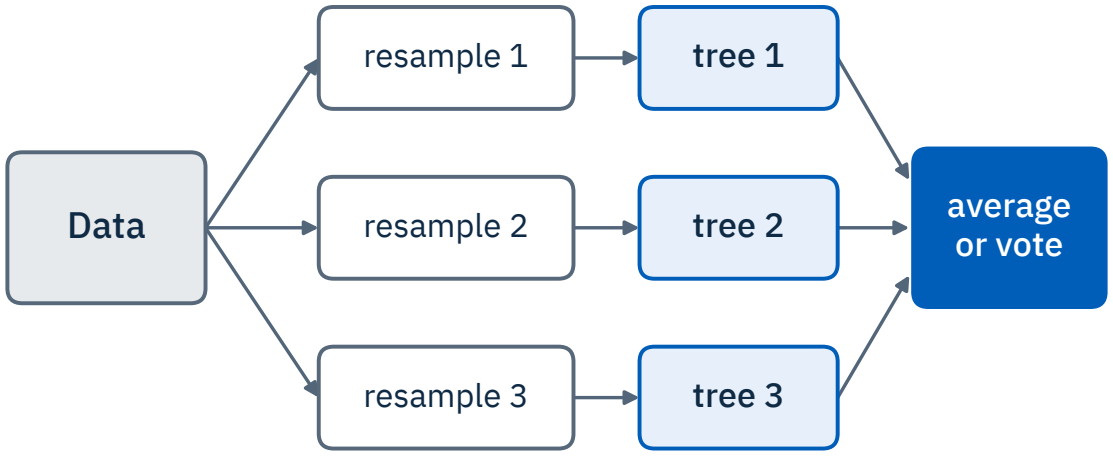
From AdaBoost to gradient boosting

Each new learner fixes what the ensemble still gets wrong.

- AdaBoost and sample weights
- Gradient boosting as gradient descent on predictions
- Losses, learning rate, and early stopping
- ROP on a new well

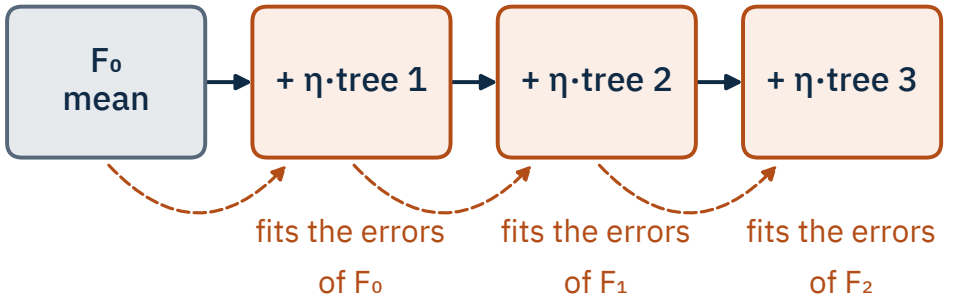
Bagging runs in parallel; boosting runs in sequence

Bagging: independent models, trained in parallel



Trees never see each other's errors.
Deep trees: low bias, averaging cuts variance.

Boosting: each model corrects the ensemble so far



$$F_3(x) = F_0 + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \eta \cdot h_3(x)$$

Shallow trees: each step is small; the sum becomes flexible.

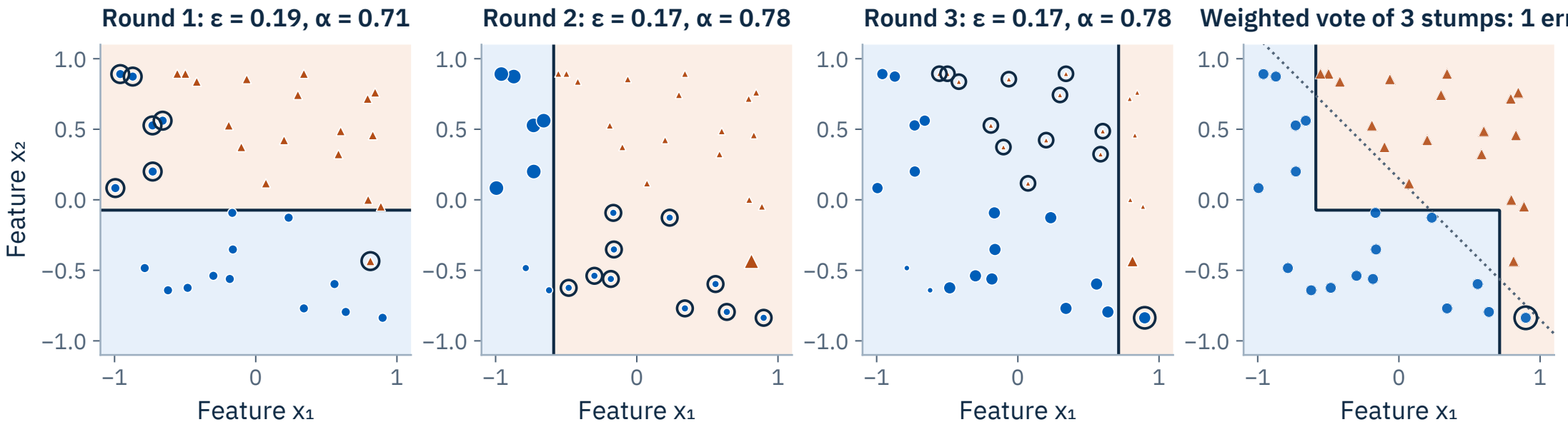
Schematic comparison of bagging and boosting.

Bagging and boosting use different training logic

Question	Bagging	Gradient boosting
How are models trained?	Separately on perturbed samples	Sequentially against current loss
What combines them?	Average or vote	Additive weighted updates
Typical base learner	Relatively deep tree	Shallow tree
Main sensitivity	Correlated trees, leaf size	Learning rate, stages, noisy targets

AdaBoost up-weights the rows it gets wrong

Marker area = row weight; rings mark rows the current stump gets wrong, which gain weight next round.



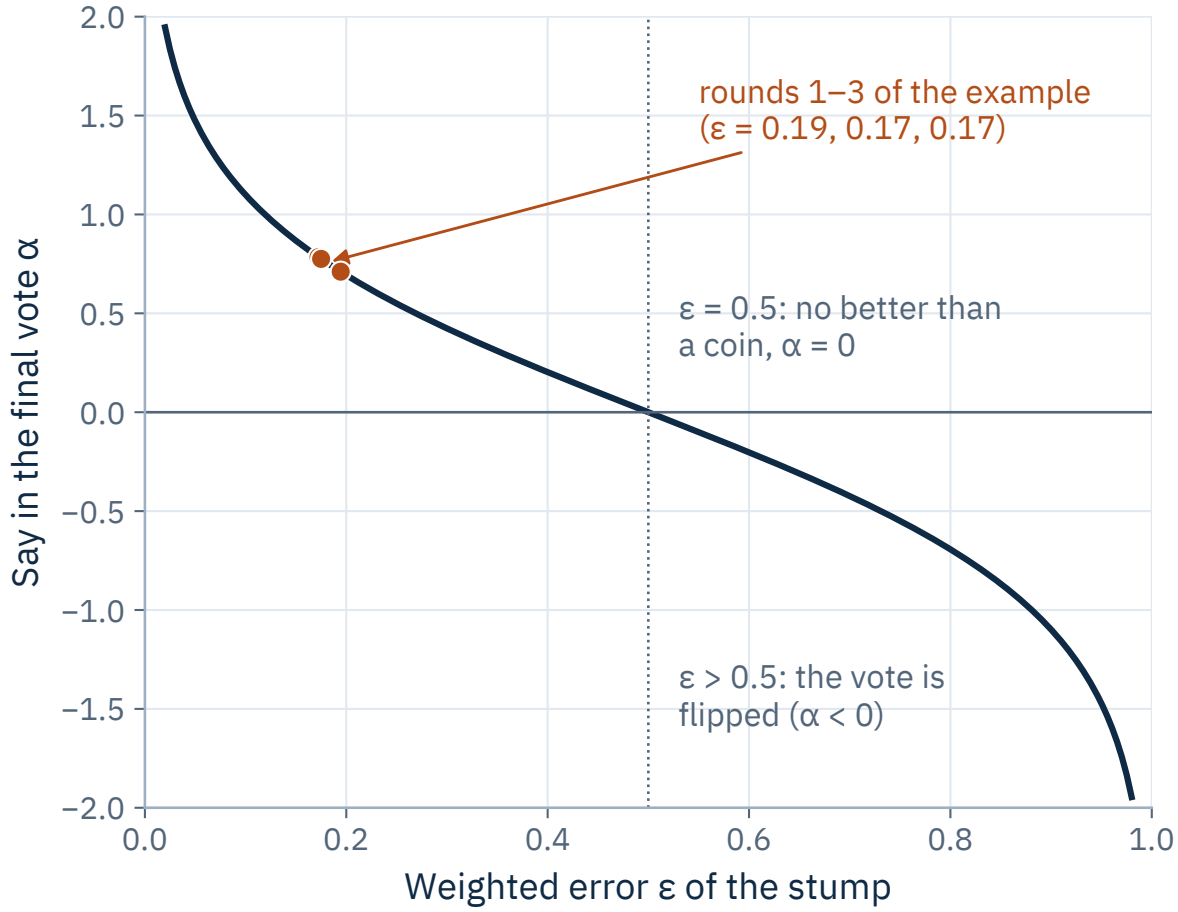
Discrete AdaBoost with decision stumps on 36 synthetic points (seed 12); dotted line: true boundary $x_1 + x_2 = 0.15$.

AdaBoost gives accurate stumps a larger say

$$\alpha_m = \frac{1}{2}\ln\frac{1-\varepsilon_m}{\varepsilon_m}$$

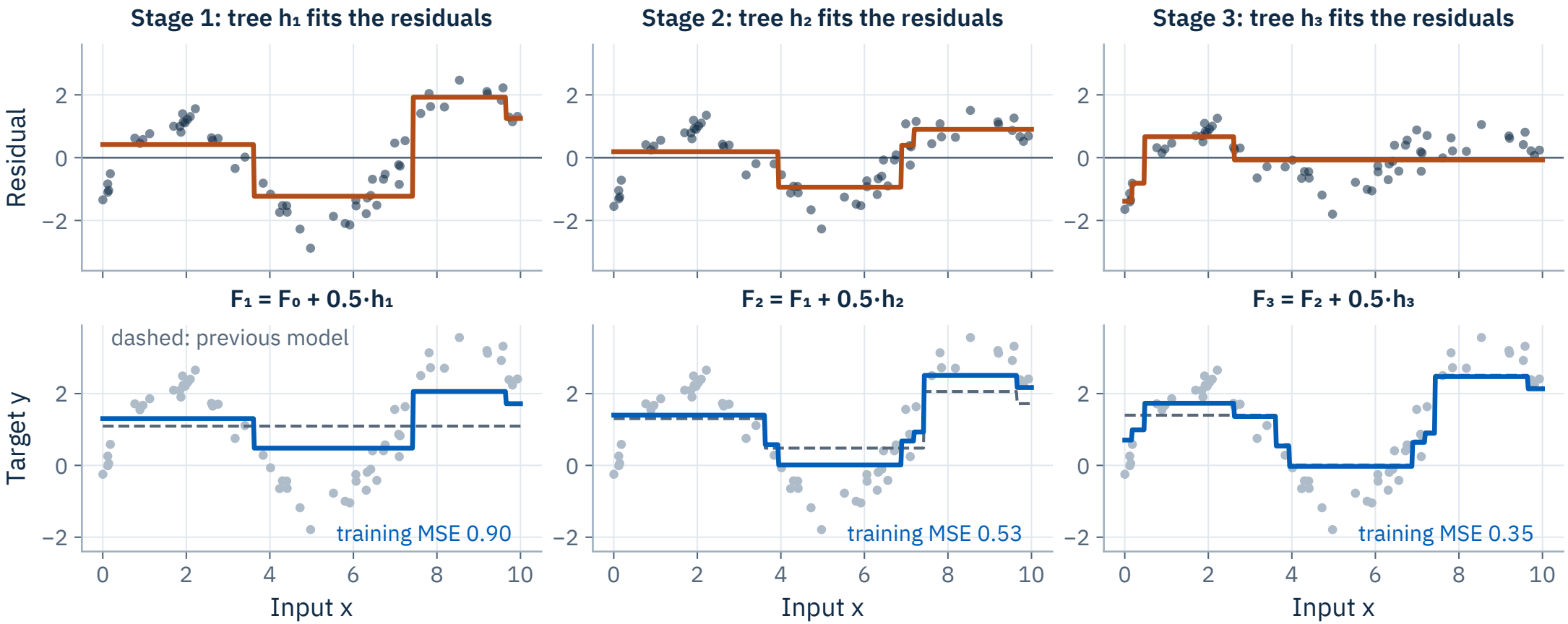
$$F(x) = \operatorname{sign}\left(\sum_m \alpha_m\, h_m(x)\right)$$

- ε_m: weighted error of stump m; weights sum to 1.
- Misclassified rows are multiplied by e^{α}, correct rows by $e^{-\alpha}$; then renormalize.
- After each update, the misclassified rows hold exactly half the weight.



Exact formula; points are the three rounds of the example.

Gradient boosting fits the residuals of the current model



Synthetic data (n = 60, seed 21); F_0 = mean of y; depth-2 regression trees; learning rate 0.5. Top: residuals and the tree fitted to them. Bottom: updated model.

Boosting is gradient descent on the predictions

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right] F = F_{m-1}$$

- Start with a constant prediction that minimizes the loss.
- Fit the next tree h_m to the negative gradients r_{im} (pseudo-residuals).
- Shrink each update by the learning rate η.
- Squared loss: r_{im} is the ordinary residual $y_i - F_{m-1}(x_i)$.

PRACTICE

5 min · Individual

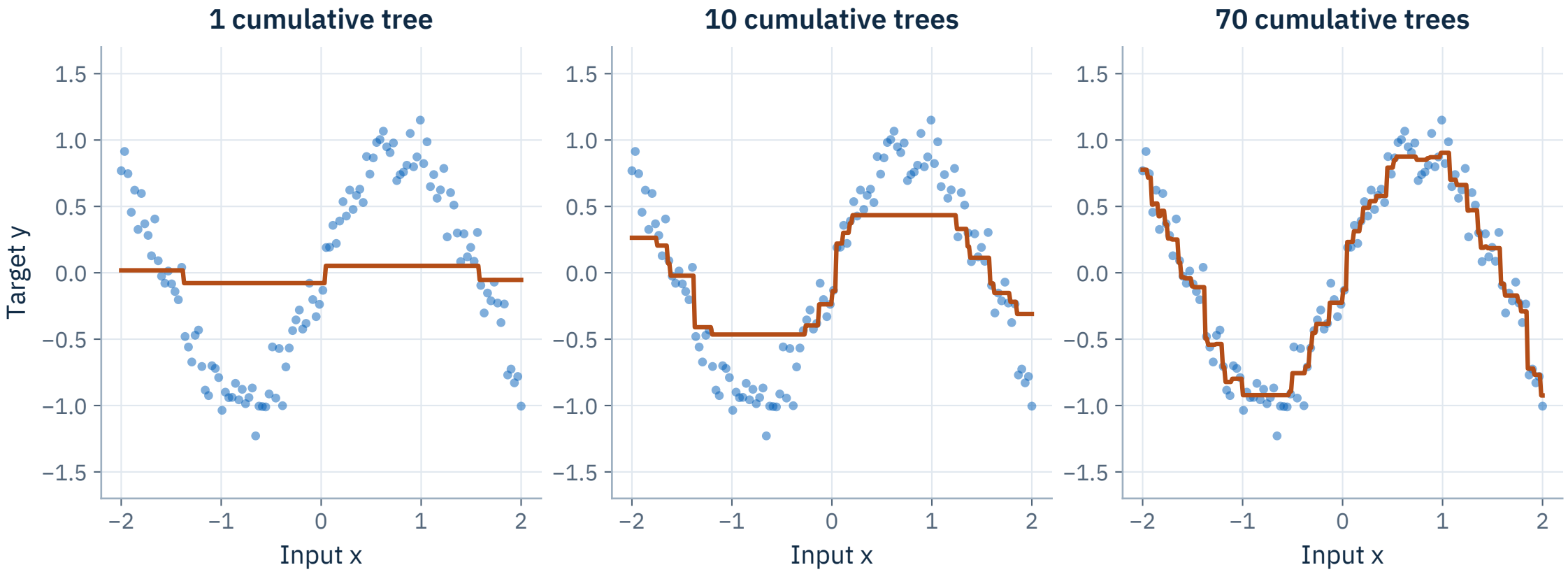
A boosting update by hand

Observed targets: [2, 4, 6]. Current predictions: [4, 4, 4].

The new tree predicts residuals [−2, 0, 2]. Learning rate = 0.1.

1. Calculate the updated predictions.
2. Compare mean squared error before and after the update.

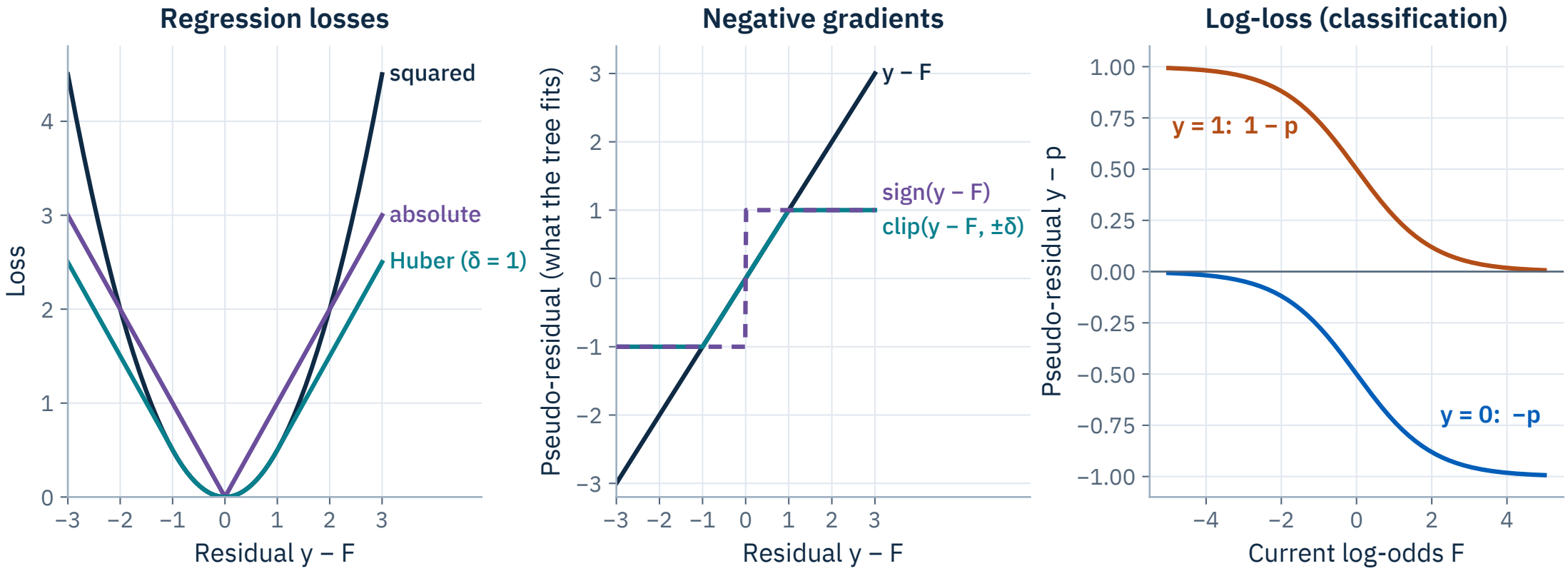
Successive stages reduce residual structure



Gradient boosting fitted to synthetic regression observations. Each panel shows the cumulative prediction after the stated number of trees.

The loss decides what each tree fits

Each tree fits the negative gradient of the loss at the current predictions: the pseudo-residual.



Exact functions. Huber threshold δ = 1; p = 1 / (1 + e^(−F)).

Each loss has its own pseudo-residual

Loss	L(y, F)	Pseudo-residual	Use when
Squared	$\frac{1}{2}(y - F)^2$	$y - F$	Default for regression
Absolute	$\lvert y - F \rvert$	$\operatorname{sign}(y - F)$	Heavy-tailed target errors
Huber (δ)	Quadratic within ±δ, linear beyond	$y - F$ clipped to ±δ	Mostly clean, some bad readings
Log-loss	$-y \ln p - (1 - y) \ln(1 - p)$	$y - p$	Binary classification (F = log-odds)

In scikit-learn: loss="squared_error", "absolute_error", "huber", or "log_loss".

PRACTICE

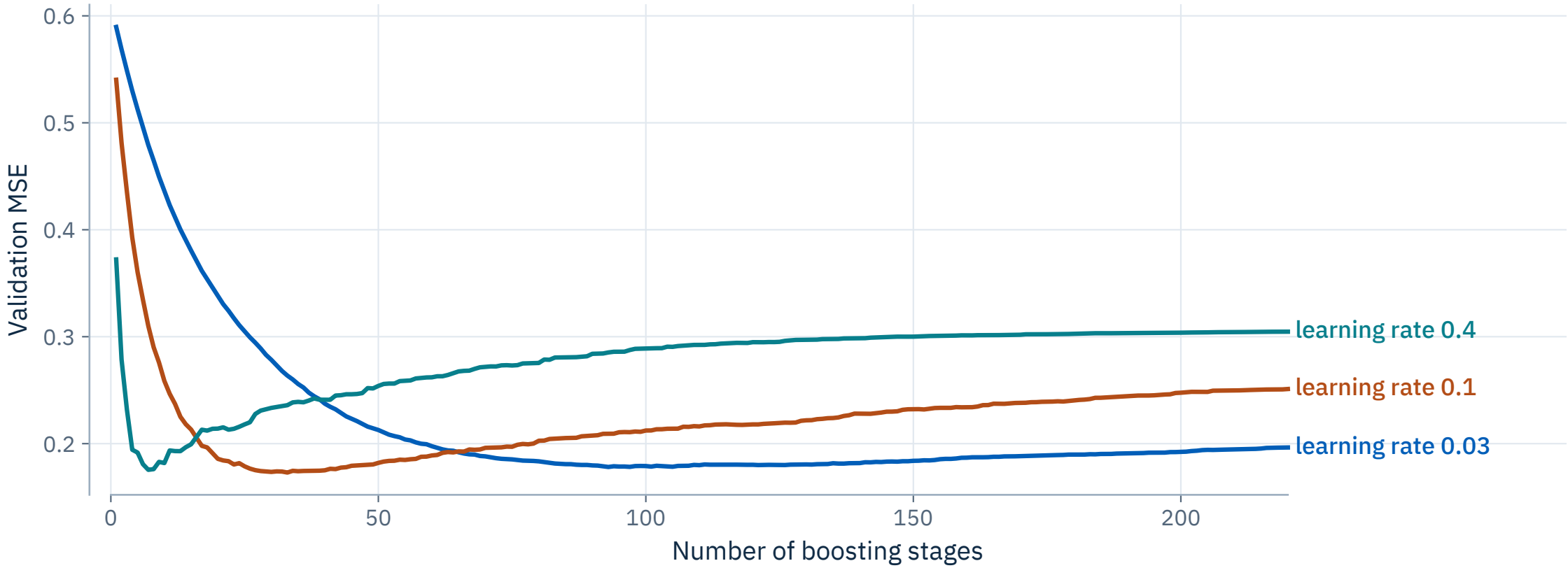
5 min · Pairs

Pseudo-residuals by hand

Current residuals y – F for four rows: −3, −0.5, 0.2, and 4.

1. Write the pseudo-residuals the next tree fits under squared, absolute, and Huber (δ = 1) loss.
2. Which loss lets the row with residual 4 dominate the next tree?
3. Log-loss: y = 1 with p = 0.8, and y = 0 with p = 0.3. What are the pseudo-residuals?

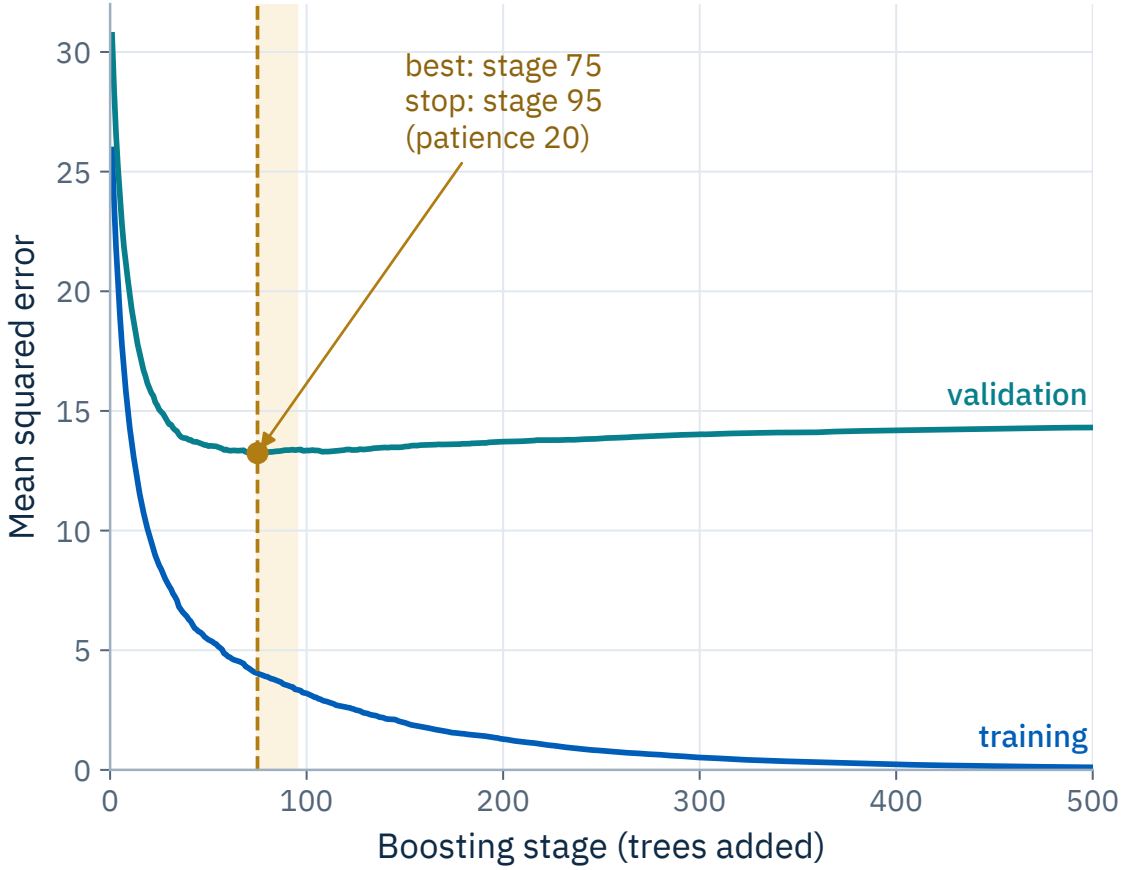
Learning rate and tree count interact



Measured validation error on a fixed synthetic holdout. Curves compare learning rates using the same data and tree depth.

Early stopping chooses the number of trees

- Hold out validation rows: another well or a later period.
- Stop after a patience of N stages without improvement.
- Keep the final test rows out of this loop.

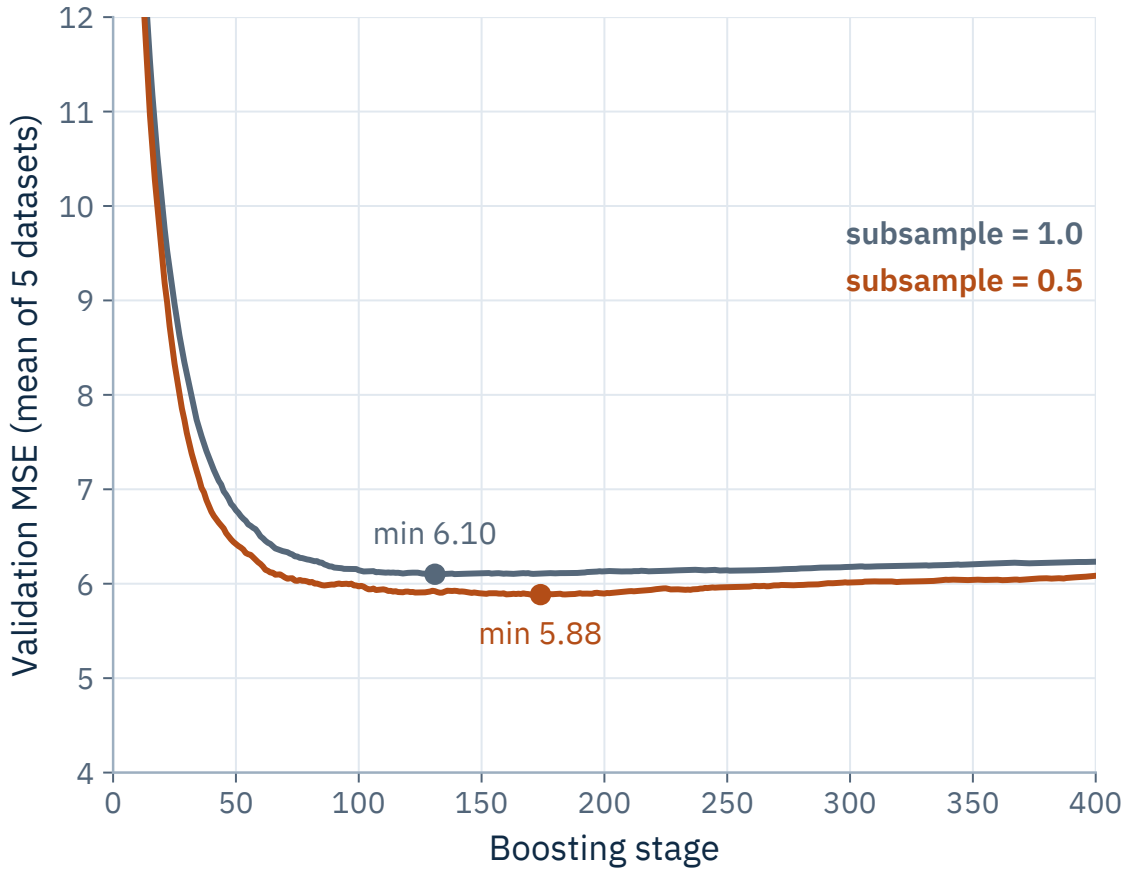


Friedman #1 synthetic regression: 400 training and 2,000 validation rows; learning rate 0.1, depth 3. Stop rule: 20 stages without improvement.

◆ Training error always falls; only validation error can say stop.

Regularization in boosting

- Limit tree depth or the number of leaves.
- Use a minimum leaf size to reduce fragile local updates.
- Row or feature subsampling reduces dependence on individual samples.
- Tune learning rate and iteration count together.



Friedman #1 synthetic regression (noise sd 2), mean of five datasets; learning rate 0.1, depth 3.

Three controls set a gradient-boosting run

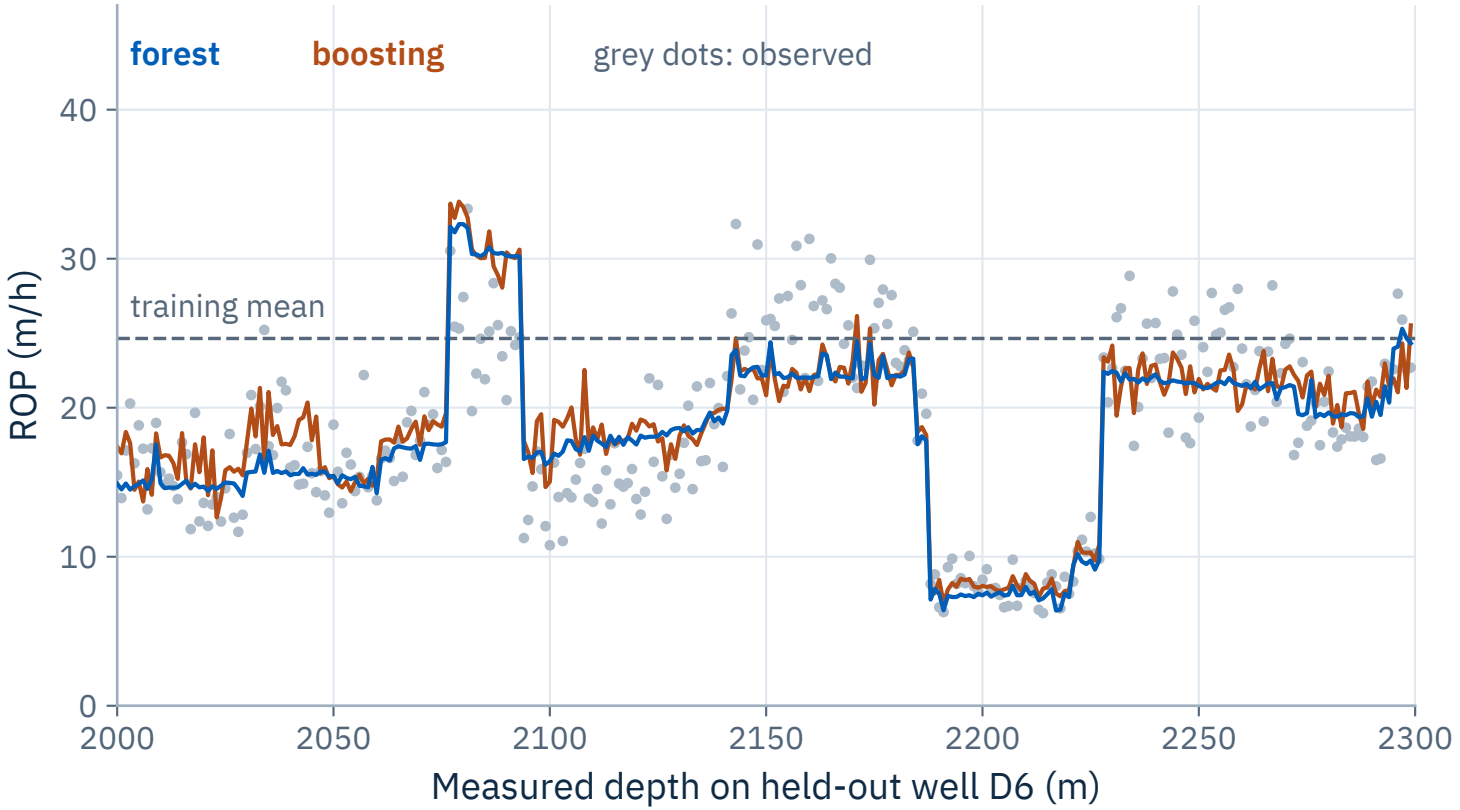
```
gbr = GradientBoostingRegressor(
    loss="huber",            # robust to bad readings
    learning_rate=0.05,      # shrinkage eta
    n_estimators=2000,       # upper limit only
    max_depth=3,             # shallow trees
    subsample=0.7,           # stochastic boosting
    validation_fraction=0.15,
    n_iter_no_change=30,
    random_state=0,
)
gbr.fit(X_train, y_train)
print("stages kept:", gbr.n_estimators_)
```

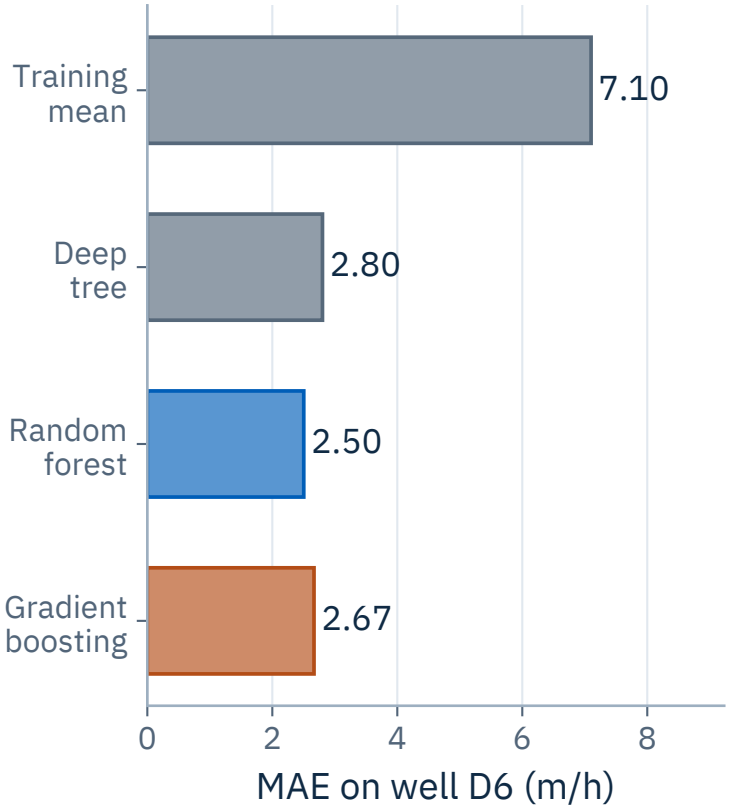
OUTPUT

```
stages kept: 173
```

- Learning rate, tree size, and stages are the core trio.
- n_estimators is a ceiling; early stopping picks the count.
- The internal validation split is random rows.

Forest and boosting score alike on a new well





Synthetic drilling wells (seed 11): trained on D1–D5, evaluated on held-out well D6. Forest: 300 trees, min_samples_leaf 3; boosting: HistGradientBoostingRegressor defaults.

KEY TAKEAWAYS

Morning takeaways before the laboratory

- ✓ Combining helps when members are useful and their errors differ.
- ✓ Bagging averages deep trees; random forests also decorrelate them with max_features.
- ✓ Out-of-bag scores assume unrelated rows; use grouped validation for new wells.
- ✓ Boosting adds shallow trees fitted to pseudo-residuals, shrunk by the learning rate.
- ✓ Early stopping on deployment-like validation data sets the number of stages.

LABORATORY

60 min

Laboratory 1: tree, forest, and boosting compared

- Parts 1–2: stratified 80/20 split (random_state 42), then a single decision tree.
- Parts 3–4: a random forest and gradient boosting on the same split.
- Part 5: 5-fold CV of all three on the training rows; then open the test rows once.
- Extension: set oob_score=True and plot a staged_predict validation curve.

NOTEBOOK

`exercises/Ensemble_Methods_Student.ipynb`

EVIDENCE TO SUBMIT

- A score table: CV mean ± sd per model
- One OOB or validation-curve plot
- One sentence: is any difference larger than fold noise?

DISCUSSION

Lab 1 debrief: the ranking is within the noise

Model	Test accuracy (30 rows)	5-fold CV on training rows
Single tree	0.933	0.942 ± 0.020
Random forest	0.900	0.950 ± 0.017
Gradient boosting	0.967	0.967 ± 0.017

- One test row is worth 3.3 points: the test ranking rests on one or two rows.
- The forest's OOB accuracy on the training rows is 0.942.

PART 4 OF 6

04

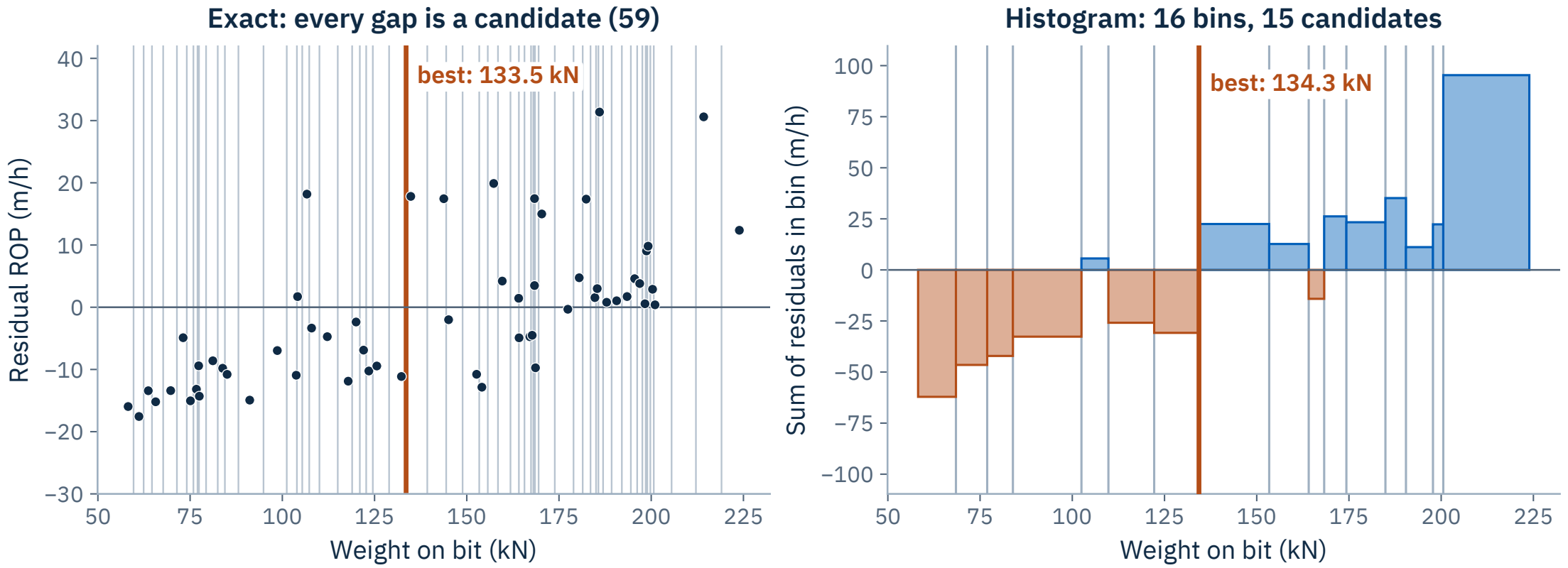
Boosting libraries and stacking

Faster trees, native gaps and categories, and learned combinations.

- Histogram binning and tree-growth policies
- Missing values and categories inside the model
- XGBoost, LightGBM, CatBoost, and scikit-learn
- Stacking with out-of-fold predictions

Histogram binning makes split search cheap

Bin each feature once; every split then needs only per-bin sums of gradients and counts.



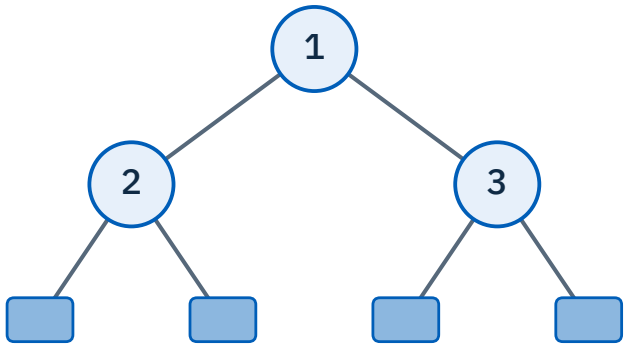
Sixty rows of synthetic drilling data (seed 4); residuals after the first-stage mean prediction; 16 quantile bins. Both searches choose the same partition.

Libraries grow trees in different orders

Same three splits, different shapes: leaf-wise trees grow deep where the gain is.

Level-wise (depth-wise)

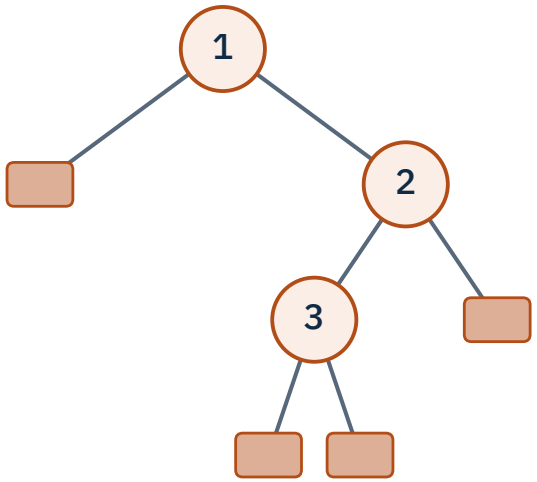
fills each level before going deeper



XGBoost default · limit: max_depth

Leaf-wise (best-first)

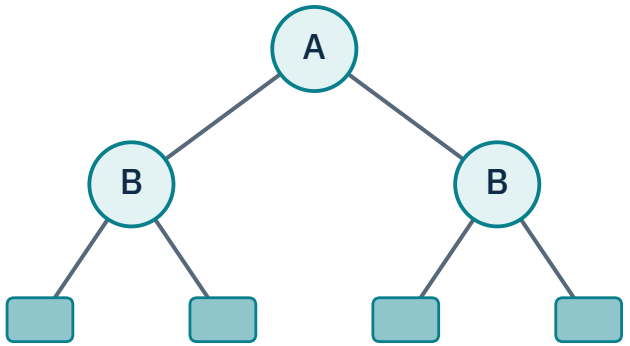
splits the leaf with the largest gain



LightGBM, sklearn HistGB · limit: leaves

Symmetric (oblivious)

one condition per level (A, then B)

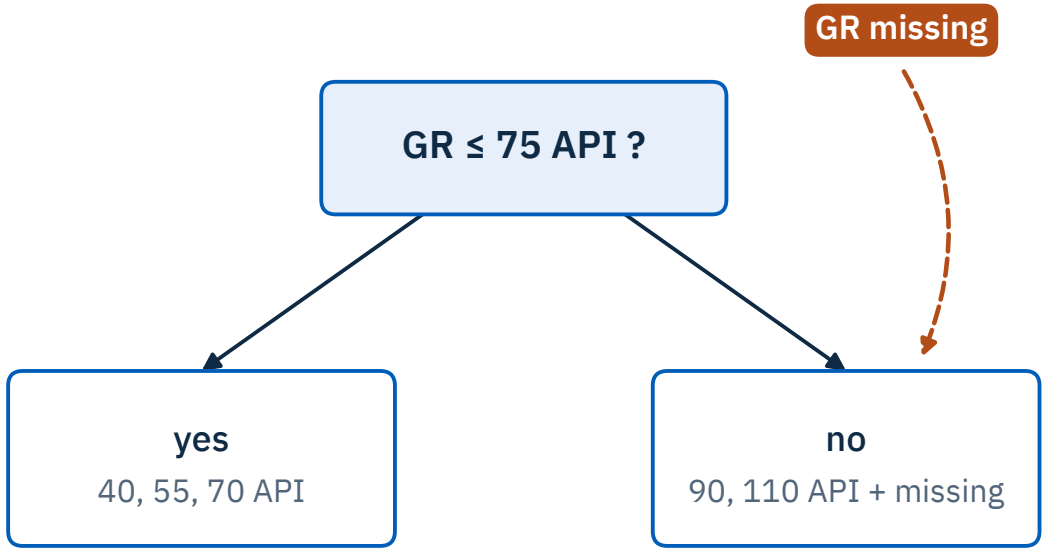


CatBoost default · limit: depth

Schematic. Numbers give the order in which splits are made; A and B mark one condition shared across a level.

Missing values get a learned direction at each split

- No imputation needed: rows with a missing value go left or right.
- The direction is the one with the larger training gain.
- Categories split as groups, not as one-hot columns.



During training both directions are scored;

the one with the larger gain is stored.

No missing GR in training? sklearn sends them to the larger child.

◆ A gap is informative only if it is missing for the same reason in production.

Schematic of native missing-value handling in histogram gradient boosting.

HistGradientBoosting handles gaps and categories natively

```
cols = ["WOB", "RPM", "FLOW", "BIT_HOURS", "FORMATION"]
names = ["Claystone", "Sandstone", "Limestone", "Dolomite"]
kinds = pd.CategoricalDtype(names)                    # fixed category list
X_tr = train[cols].astype({"FORMATION": kinds})
X_va = valid[cols].astype({"FORMATION": kinds})       # a validation well
print(X_tr.isna().mean().round(3).to_dict())

hgb = HistGradientBoostingRegressor(
    max_iter=1000, learning_rate=0.1, early_stopping=True,
    n_iter_no_change=20, random_state=0)
hgb.fit(X_tr, train["ROP"], X_val=X_va, y_val=valid["ROP"])   # sklearn 1.7
print("iterations used:", hgb.n_iter_)
```

OUTPUT

```
{'WOB': 0.0, 'RPM': 0.014, 'FLOW': 0.062, 'BIT_HOURS': 0.0, 'FORMATION': 0.0}
iterations used: 160
```

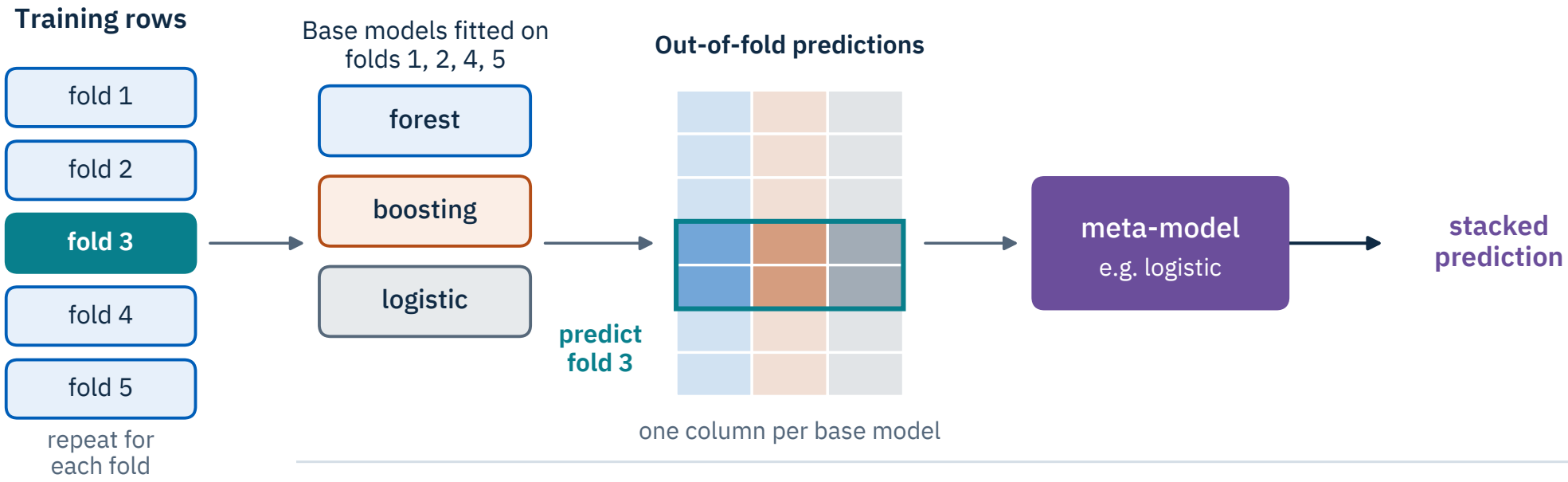
Four libraries, one idea, different defaults

Library	Default tree growth	Categorical inputs	Missing values
XGBoost	Level-wise (max_depth)	enable_categorical=True	Learned default direction
LightGBM	Leaf-wise (num_leaves)	categorical_feature	Learned direction
CatBoost	Symmetric (oblivious)	Ordered target statistics	Numeric NaN below minimum
scikit-learn HistGB	Leaf-wise (max_leaf_nodes)	categorical_features	Learned direction

No universal ranking: compare on the same split, metric, hardware, and tuning budget.

Stacking needs out-of-fold predictions

The meta-model must learn from predictions on rows each base model did not fit.



Schematic of stacking with K = 5 folds and three base models.

StackingClassifier builds the out-of-fold features

```
members = [("forest", RandomForestClassifier(300, random_state=0)),
           ("hgb", HistGradientBoostingClassifier(random_state=0)),
           ("logit", make_pipeline(StandardScaler(), LogisticRegression()))]
stack = StackingClassifier(members, final_estimator=LogisticRegression(),
                           cv=5, stack_method="predict_proba")

for name, model in members + [("stack", stack)]:
    s = cross_val_score(model, X, y, groups=wells, cv=GroupKFold(4))
    print(f"{name:7s} {s.mean():.3f}")
```

OUTPUT

```
forest  0.776
hgb     0.820
logit   0.857
stack   0.852
```

⚠ The internal cv=5 splits rows, not wells: its out-of-fold columns still leak within wells.

PRACTICE

5 min · Groups of 3

Choosing forest or boosting

A tabular model will run at a remote wellsite. You have limited tuning time.

1. Which baseline would you try first, and why?
2. What comparison would make you change your choice?
3. What must fit within the deployment budget?

PART 5 OF 6

05

Interpreting ensemble models

What did the model learn, and how far can we trust the explanation?

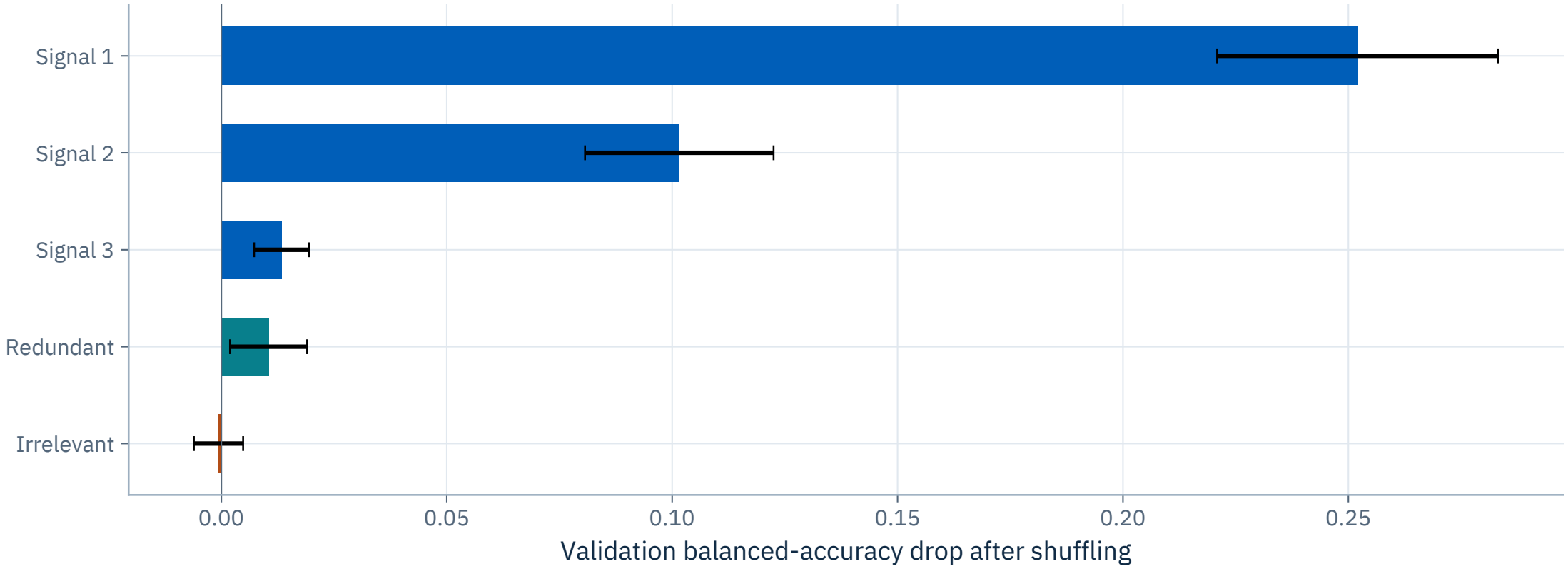
- Impurity and permutation importance
- Partial dependence and ICE curves
- Shapley values, computed exactly
- Cautions: association, extrapolation, stability

Feature importance depends on the question

Method	Measures	Main limitation
Split impurity importance	Training impurity reduction credited to a feature	Can favour high-cardinality predictors
Permutation importance	Evaluation score drop after shuffling a feature	Correlated substitutes can hide reliance
Domain sensitivity review	Plausibility of model response to input changes	Requires careful constraints

Importance can expose an irrelevant feature

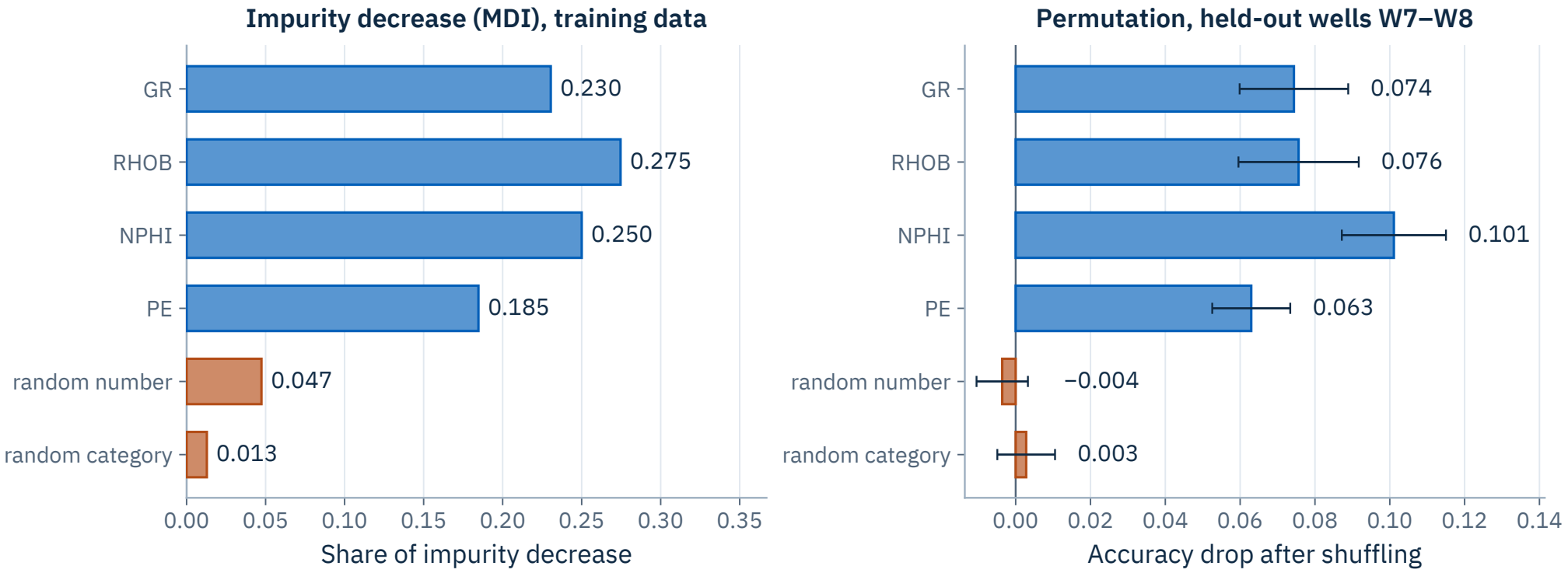
Shuffle one column in the validation rows; the score drop measures the model's reliance on it.



Computed synthetic validation experiment with a deliberately irrelevant feature. Error bars show variation over repeated shuffles.

Impurity importance rewards high-cardinality noise

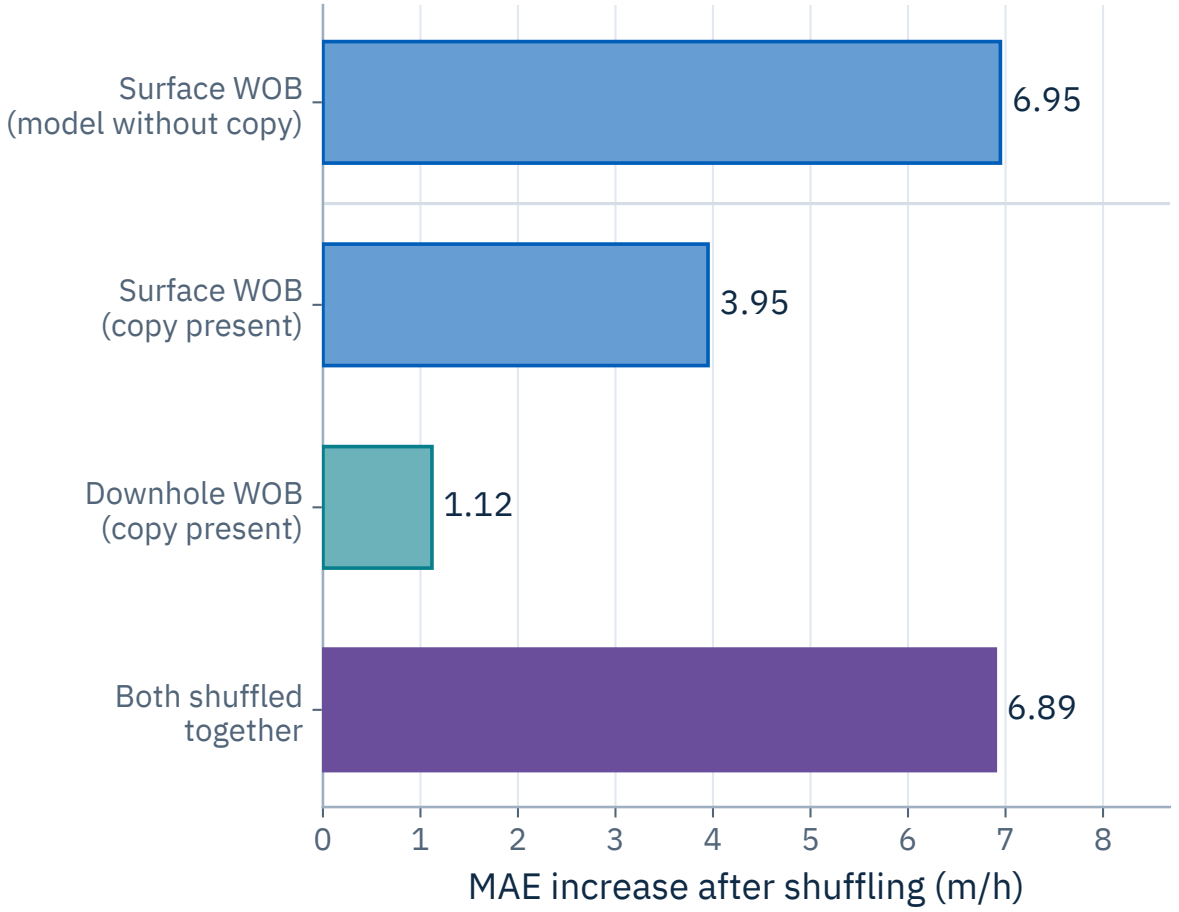
Random numbers get a quarter of PE's impurity credit, yet shuffling them changes nothing on new wells.



Synthetic facies logs plus two pure-noise columns (seed 0); forest of 300 unpruned trees trained on W1–W6; permutation on W7–W8, 20 repeats.

Correlated features share and hide importance

- Downhole WOB is a near copy of surface WOB (r = 0.997).
- With both present, shuffling one leaves the other: 3.95 + 1.12.
- Shuffling them together recovers the joint effect: 6.89.



◆ Group correlated inputs before ranking them.

Synthetic drilling data; forest trained on D1–D4, permutation on D5–D6, 30 repeats.

Importance and dependence plots take a few lines

```python
from sklearn.inspection import permutation_importance
from sklearn.inspection import PartialDependenceDisplay

r = permutation_importance(model, X_val, y_val, n_repeats=20, random_state=0,
                           scoring="neg_mean_absolute_error")
for name, m, s in zip(X_val.columns, r.importances_mean, r.importances_std):
    print(f"{name:10s} {m:5.2f} ± {s:.2f}")

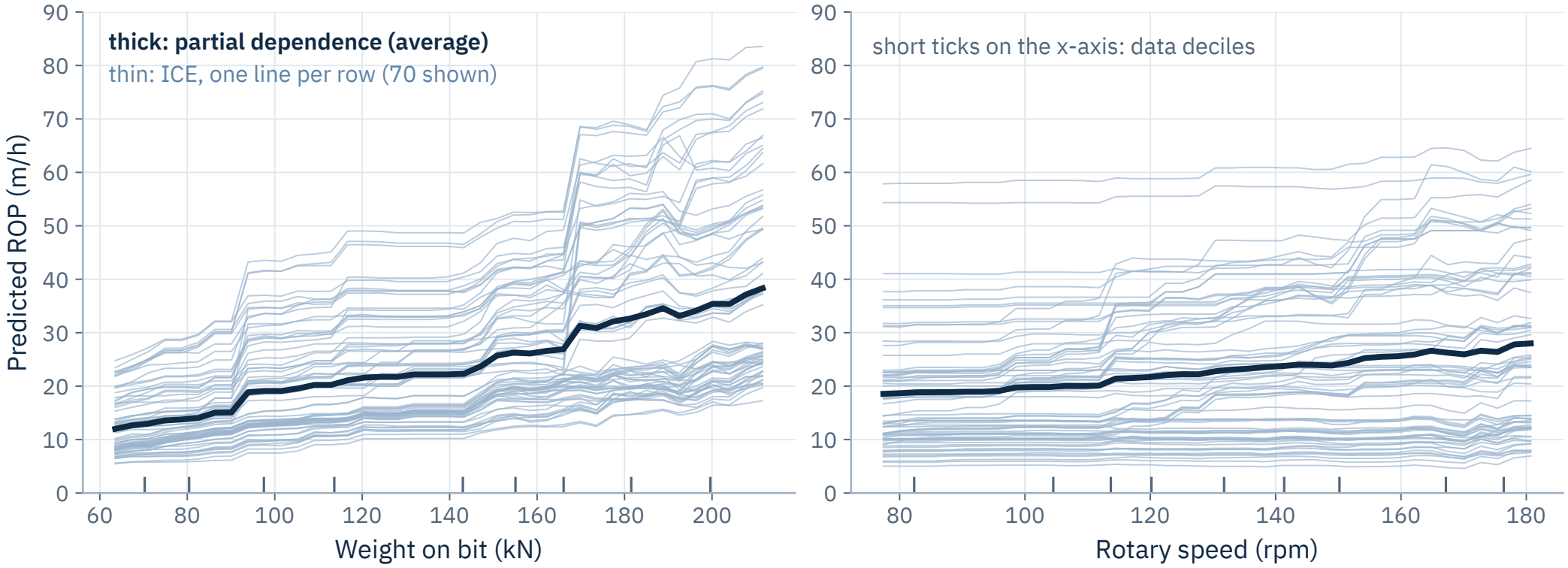
PartialDependenceDisplay.from_estimator(
    model, X_val, features=["WOB", "RPM"], kind="both",
    subsample=70, random_state=0)
```

OUTPUT

```
WOB         6.87 ± 0.33
RPM         1.83 ± 0.13
FLOW        0.48 ± 0.04
BIT_HOURS   0.13 ± 0.03
FORMATION   8.99 ± 0.39
```

Partial dependence averages the model's response

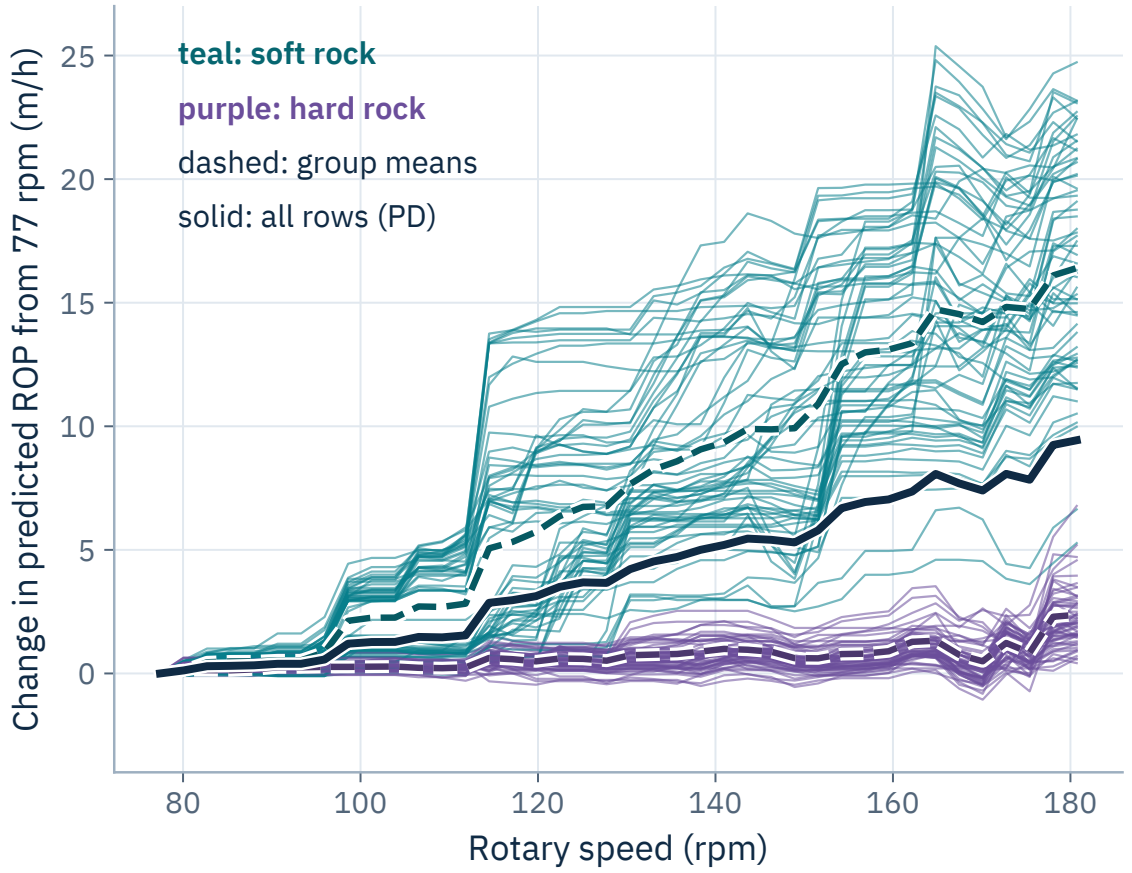
PD: set WOB to each grid value for every row, predict, and average. ICE: keep one line per row.



HistGradientBoostingRegressor trained on synthetic wells D1–D5; sklearn.inspection.partial_dependence over 300 sampled rows, 40 grid points between the 5th and 95th percentiles.

ICE curves reveal what the average hides

- Centre each curve at the lowest rpm to compare slopes.
- Soft rock: predicted ROP rises with rpm; hard rock: almost flat.
- The average curve describes neither group.

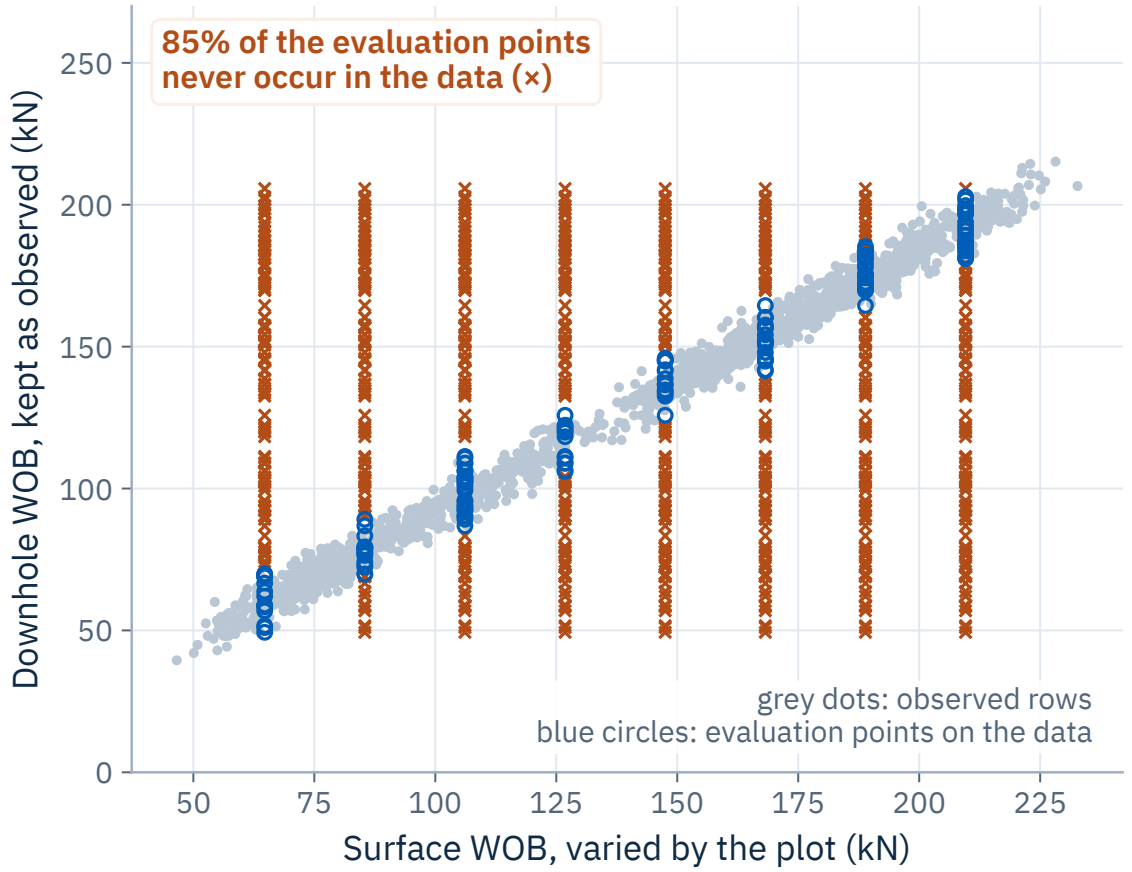


Centred ICE curves from the synthetic-drilling boosting model; 120 of 300 rows shown, coloured by formation.

◆ Diverging ICE curves signal an interaction: plot them by group.

Partial dependence can query impossible inputs

- PD pairs each grid value with every observed value of the other inputs.
- With correlated inputs most pairs never occur: 85% here.
- The model's answers there are extrapolation, not evidence.

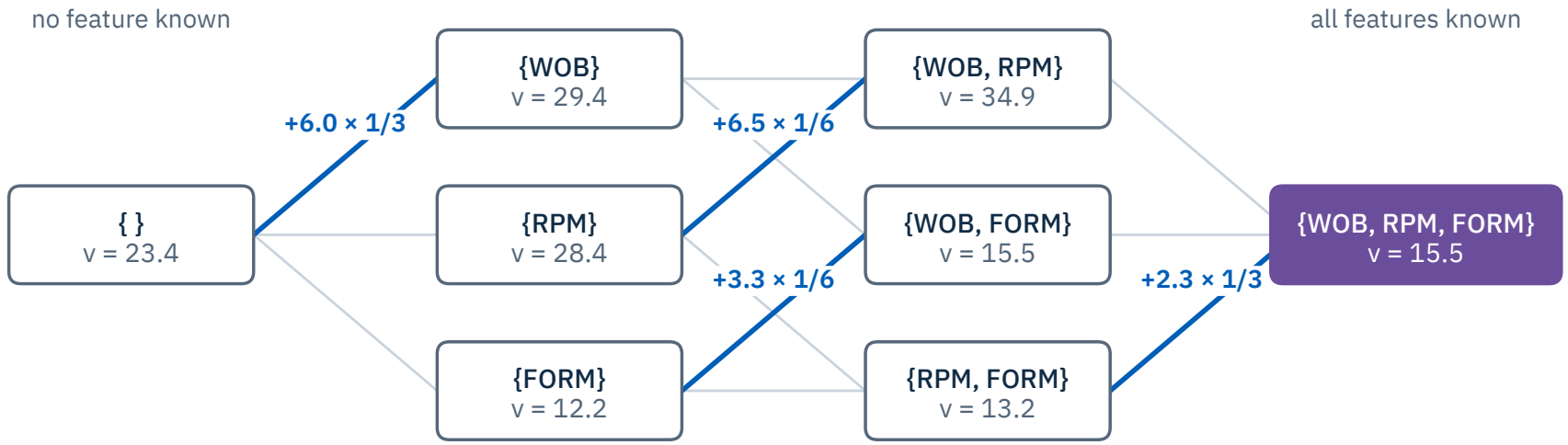


Synthetic drilling data with a downhole WOB column; 8 grid values × 120 observed downhole values.

◆ Restrict to observed combinations, or use accumulated local effects.

Shapley values average a feature's marginal steps

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\ (p - |S| - 1)!}{p!} \left[v(S \cup \{j\}) - v(S) \right]$$



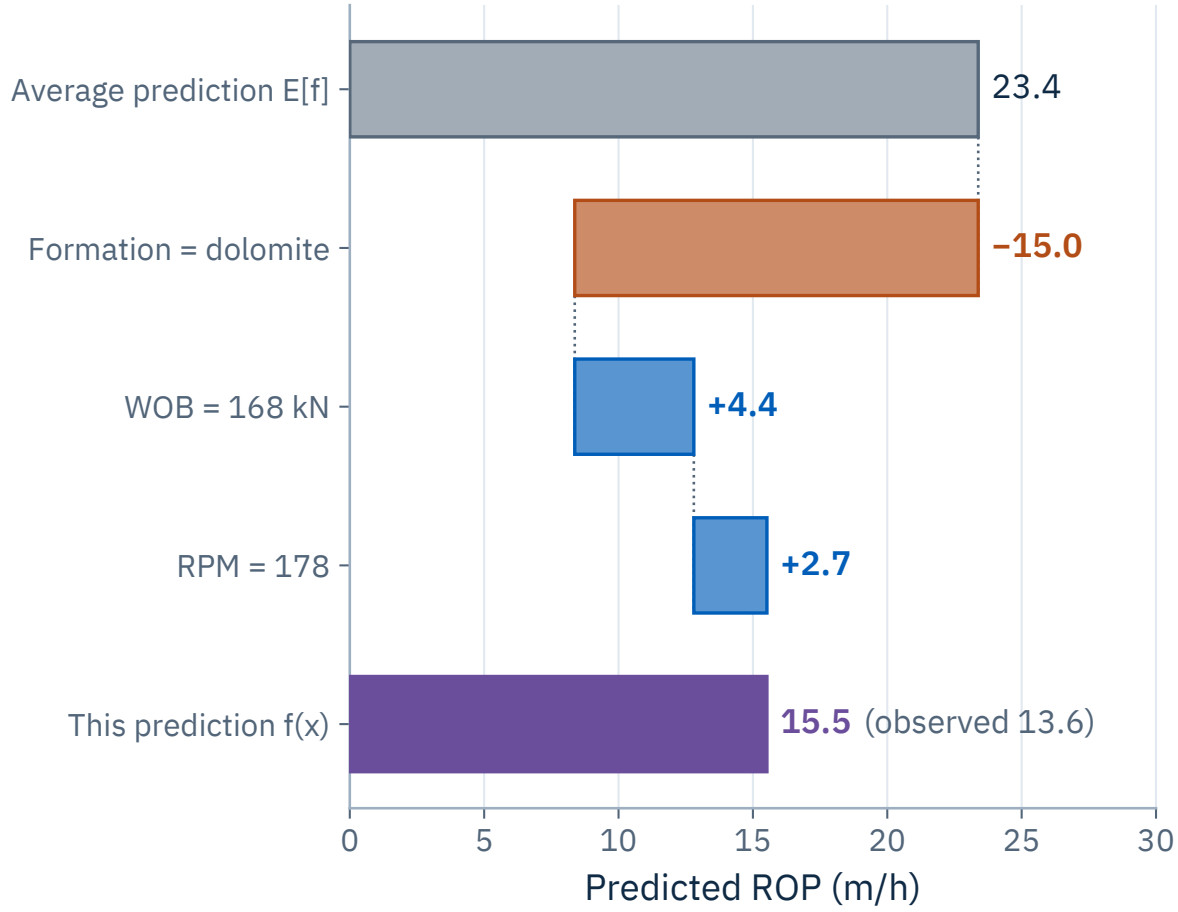
φ(WOB) = weighted sum of the four blue steps = +4.43 m/h · v(S): mean prediction with the features in S fixed to this row

Exact computation for one row of well D6; 3-feature boosting model; v(S) in m/h over 200 background rows.

Shapley values add up to the prediction

$$f(x) = \mathrm{E}[f(X)] + \sum_{j=1}^{p} \phi_j$$

- Base: the average prediction over the background rows, 23.4 m/h.
- Dolomite –15.0, WOB +4.4, RPM +2.7: this row predicts 15.5 m/h.
- Features outside S are filled from background rows (interventional).



Exact interventional Shapley values for one row of synthetic well D6.

PRACTICE

5 min · Pairs

Shapley values by hand

A two-feature ROP model gives average predictions (m/h): no feature known 20; WOB known 26; RPM known 22; both known 30.

1. Compute the Shapley value of WOB and of RPM.
2. Check that they add up to the prediction minus the base.
3. Why is WOB's step larger once RPM is known?

Choose the explanation that answers your question

Method	Question it answers	Scope	Main pitfall
Impurity (MDI)	Which splits cut training impurity?	Global, training	Rewards high-cardinality noise
Permutation	How much does the score rely on a feature?	Global, held-out	Correlated inputs share credit
PD and ICE	How do predictions change with one input?	Average and per row	Extrapolates with correlated inputs
Shapley values	How is one prediction split among inputs?	One row	Depends on the background data

Explanations describe the model, not the rock

Association, not causation

- Importance shows what the model uses
- Controlled tests show what the rock does

Extrapolation

- PD and Shapley query unseen combinations
- Check the data support first

Correlated inputs

- Credit is split or hidden
- Group copies before ranking

Stability

- Recompute on other folds and wells
- Unstable rankings are not findings

Background choice

- Shapley values move with the reference rows
- State the background in every report

Leakage shows up here too

- A leaky feature often ranks first
- Treat a surprise winner as a bug report

◆ Explanations are evidence about the fitted model; decisions need domain review.

PART 6 OF 6

06

Pipelines, leakage, and deployment

The deployed predictor is the whole pipeline, with its evidence.

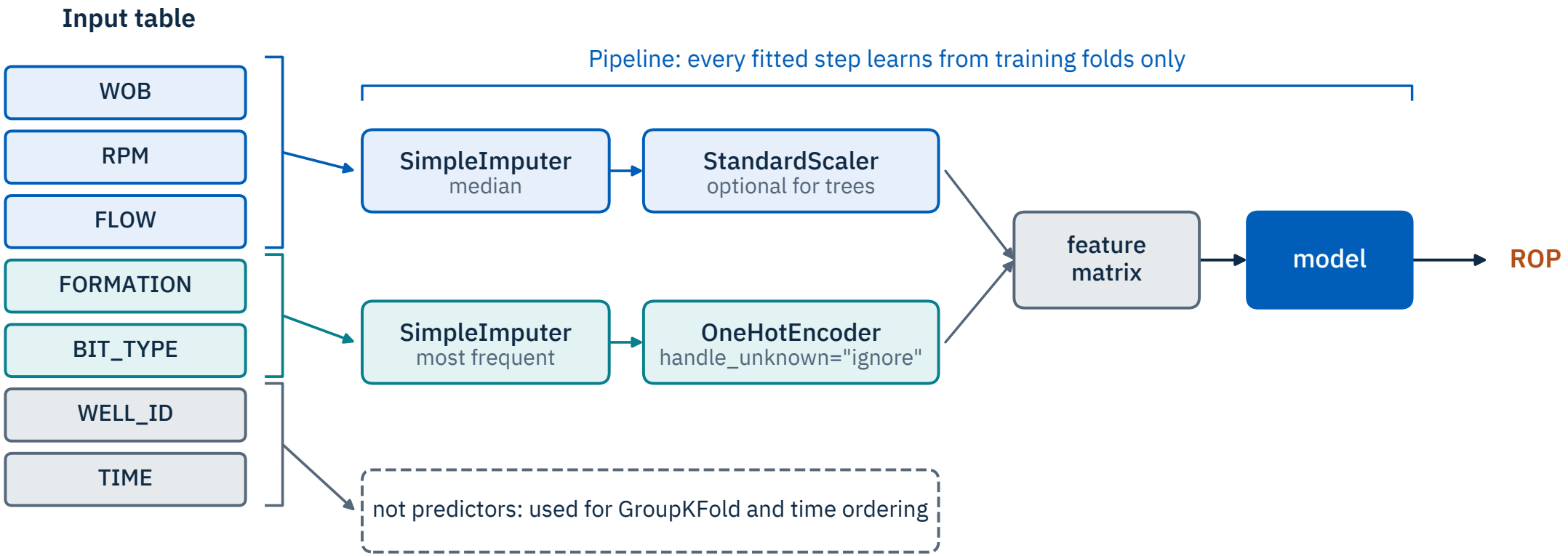
- ColumnTransformer for mixed inputs
- A leakage demonstration
- Saving an artifact with its context
- Model cards and monitoring

Mixed data needs separate transformations

Input family	Example	Training transformation
Numeric	Pressure, flow, gamma ray	Imputation and optional scaling
Categorical	Formation or equipment type	Imputation and category encoding
Metadata	Well ID, timestamp, record ID	Split, join, or ordering key

New categories and missing sensors should have defined behaviour.

A ColumnTransformer routes each column family



Schematic of a ColumnTransformer inside a Pipeline for the synthetic drilling table; column names are illustrative.

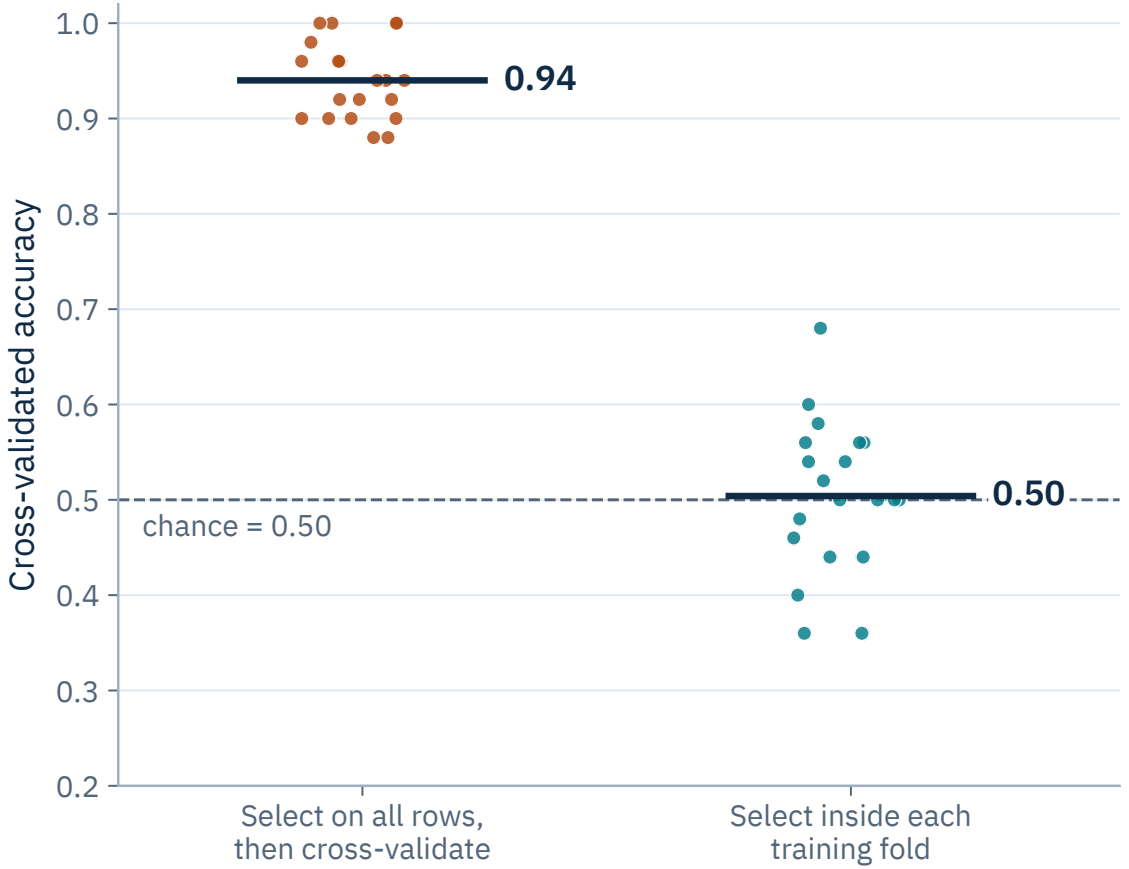
A ColumnTransformer preserves the schema

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline

numeric = make_pipeline(SimpleImputer(strategy="median"))
categorical = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore")
)
prep = ColumnTransformer([
    ("num", numeric, numeric_columns),
    ("cat", categorical, categorical_columns)
])
```

Selecting features on all rows fakes a signal

- 50 rows, 2,000 noise features, random labels: nothing to learn.
- Select the 20 "best" features on all rows, then cross-validate: 0.94.
- Select inside each training fold: 0.50, which is chance.



◆ Any step that looks at labels must sit inside the pipeline.

20 synthetic datasets (seed 0), SelectKBest(f_classif, k = 20) and logistic regression; 5-fold stratified CV.

Feature selection belongs inside the pipeline

```
rng = np.random.default_rng(0)
X = rng.normal(size=(50, 2000))                # 2,000 pure-noise features
y = rng.permutation(np.repeat([0, 1], 25))     # random labels
cv = StratifiedKFold(5, shuffle=True, random_state=0)

# Wrong: the selector sees every row, including each validation fold
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(LogisticRegression(), X_sel, y, cv=cv)

# Right: selection is refitted inside every training fold
pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression())
right = cross_val_score(pipe, X, y, cv=cv)
print(f"wrong {wrong.mean():.2f}   right {right.mean():.2f}")
```

OUTPUT

```
wrong 0.90   right 0.36
```

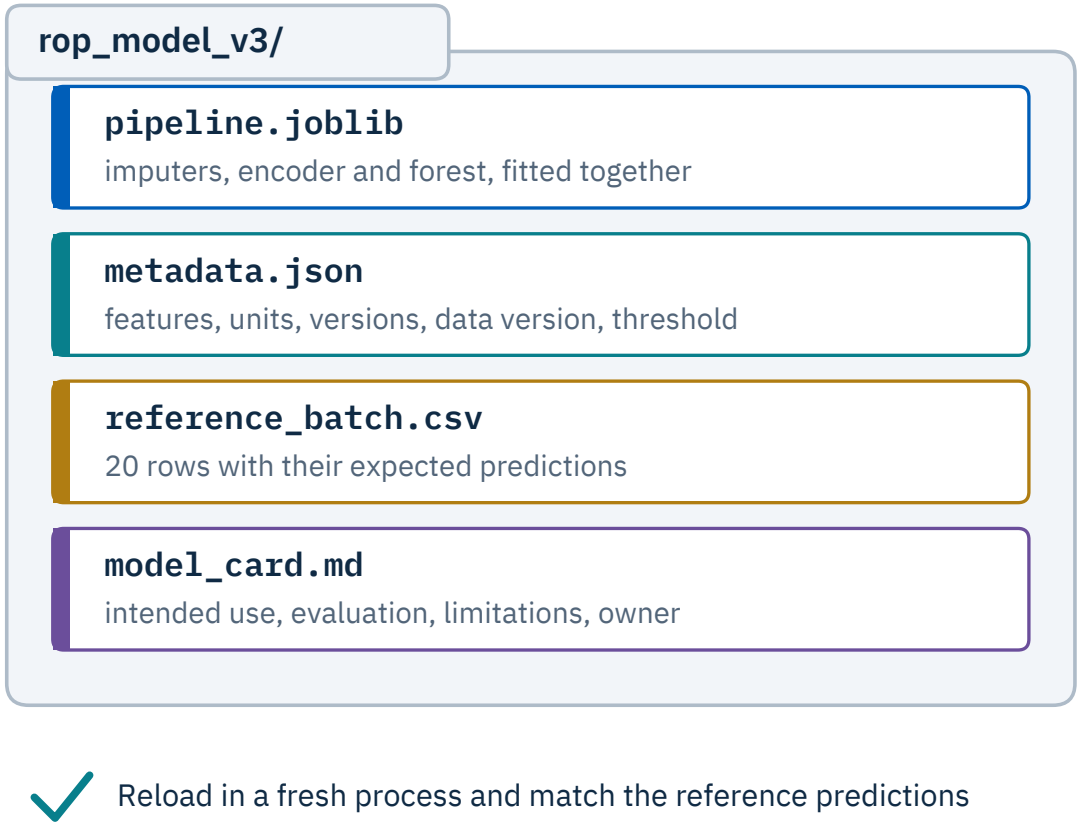
Tuning operates on the complete pipeline

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("prep", prep),
    ("forest", RandomForestClassifier(
        n_estimators=200, random_state=42, n_jobs=-1))
])
param_grid = {
    "forest__min_samples_leaf": [1, 5, 15],
    "forest__max_features": ["sqrt", 0.7]
}
```

The saved artifact needs context

- Save fitted preprocessing and estimator together.
- Record feature names, units, versions, data version, and threshold.
- Reload in a fresh process and compare predictions on a reference batch.
- Load serialized Python objects only from trusted sources.



Schematic of a versioned model artifact; file names are illustrative.

Persist the pipeline and pin the versions

```python
import json, sys, joblib, sklearn, numpy as np

joblib.dump(model, "rop_model_v3/pipeline.joblib")
meta = {
    "features": numeric_columns + categorical_columns,
    "target": "ROP (m/h)",
    "sklearn": sklearn.__version__,
    "numpy": np.__version__,
    "python": sys.version.split()[0],
    "training_data": "rop_wells_2026-09 (wells D1-D5)",
}
with open("rop_model_v3/metadata.json", "w") as f:
    json.dump(meta, f, indent=2)

reloaded = joblib.load("rop_model_v3/pipeline.joblib")   # trusted files only
assert np.allclose(reloaded.predict(X_ref), expected_ref)
```

A model card states what the model is for

Intended use

- Predict ROP on new wells in field X
- Planning aid, not an autodriller input

Training data

- Wells D1–D5, 2026 campaign, 1 m rows
- Formations covered; known gaps

Evaluation

- Grouped CV by well; MAE per formation
- Baseline: training mean and offset wells

Limitations

- No extrapolation beyond seen WOB and RPM
- Untested in formations absent from training

Owner and review

- Named owner, review every 3 months
- Retraining and rollback triggers

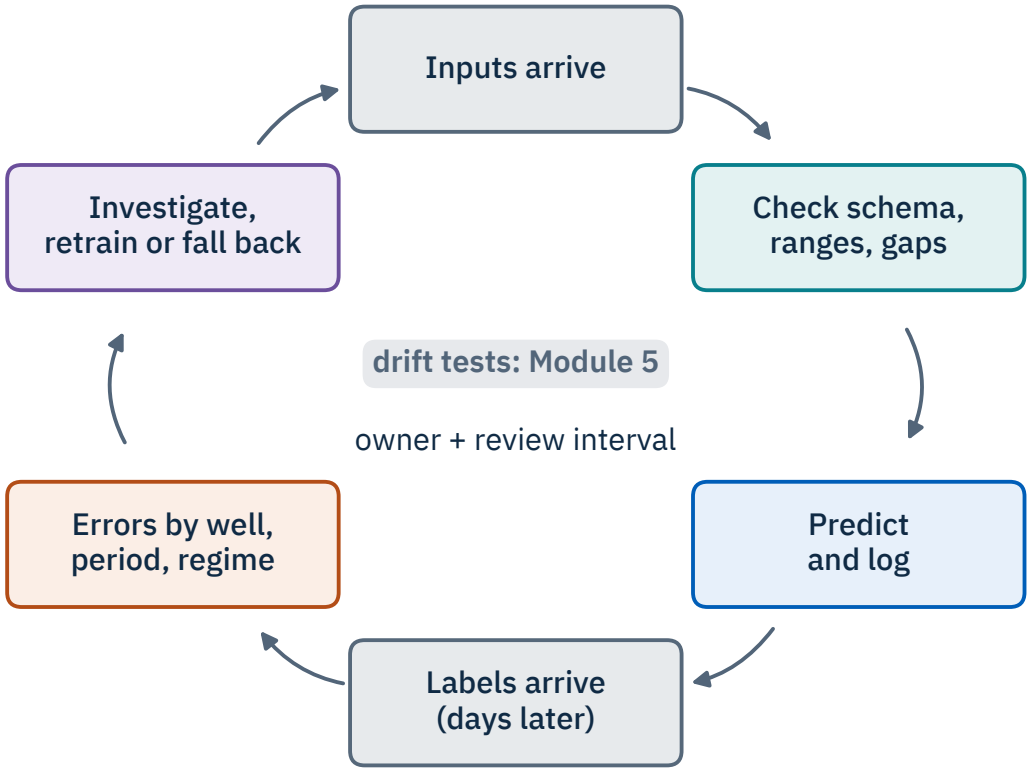
Version

- Artifact hash, library versions, date
- Link to metadata and reference batch

◆ If you cannot fill a card, the model is not ready to hand over.

Production evaluation continues after release

- Compare incoming sensor distributions with the training population.
- Track missingness, new categories, latency, and rejected inputs.
- When labels arrive, inspect errors by well, period, and operating regime.
- Define when to investigate, retrain, or fall back to a simpler method.



Schematic of a monitoring loop; drift tests are covered in Module 5.

PRACTICE

5 min · Pairs

Spot the leaks in this workflow

```
logs = pd.read_csv("facies_logs.csv")          # 8 wells, depth-ordered rows
logs["GR_z"] = (logs.GR - logs.GR.mean()) / logs.GR.std()
logs["SAND_SHARE"] = logs.groupby("WELL").FACIES.transform(
    lambda f: (f == "sandstone").mean())
sel = SelectKBest(f_classif, k=4).fit(logs[cols], logs.FACIES)
X = logs[cols].loc[:, sel.get_support()]
X_tr, X_te, y_tr, y_te = train_test_split(X, logs.FACIES, test_size=0.2)
search = GridSearchCV(RandomForestClassifier(), grid, cv=5).fit(X_tr, y_tr)
print("test accuracy:", search.score(X_te, y_te))
```

1. Find at least four problems and say how each inflates the score.
2. Rewrite the workflow as one pipeline with a grouped split.

LABORATORY

55 min

Laboratory 2: importance, pipelines, and tuning

- Part 6: compare the forest's MDI with permutation importance on the test rows.
- Part 7: build the pipeline, save it with joblib, reload it, and match predictions.
- Part 8: grid-search the pipeline and inspect every row of cv_results_.
- Part 9 (optional): XGBoost if installed, otherwise HistGradientBoostingClassifier.

NOTEBOOK

`exercises/Ensemble_Methods_Student.ipynb`

EVIDENCE TO SUBMIT

- MDI versus permutation table with one sentence of interpretation
- A reload check passing on five reference rows
- The grid-search table and your reading of the best setting

DISCUSSION

Lab 2 debrief: impurity and permutation disagree on Iris

Feature	MDI (training)	Permutation (test, 20 repeats)
Petal width	0.437	0.137
Petal length	0.431	0.128
Sepal length	0.116	−0.052
Sepal width	0.015	−0.033

- Grid search: all 18 settings tie at 0.950, so best_params_ is simply the first in the grid.
- The reloaded pipeline must reproduce predictions exactly; the scaler did not matter.

PRACTICE

5 min · Groups of 3

Evidence review

The best validation score improves by 0.002 after testing 150 settings.

1. What evidence would support calling this an improvement?
2. What uncertainty comes from the search itself?
3. How would you choose if the larger model costs ten times more to serve?

Key takeaways

- ✓ Ensembles help when members are useful and their errors differ; correlation sets the floor.
- ✓ Forests average decorrelated deep trees; boosting adds shallow trees fitted to pseudo-residuals.
- ✓ Out-of-bag scores and random validation fractions assume unrelated rows: validate by well or time.
- ✓ Explanations describe a fitted model on given data: use held-out permutation, check support, group copies.
- ✓ Ship the whole pipeline with its metadata, a reference batch, a model card, and an owner.

PRACTICE

7 min · Individual

Exit check

1. Explain the role of diversity in a forest, using the variance formula.
2. Explain what the next boosting learner fits under squared loss.
3. Explain why saving only the estimator may be insufficient.
4. Name one reason an out-of-bag score can be optimistic for a new well.
5. Give one reason impurity importance can rank a noise feature highly.

References and further study

- Breiman (1996) Bagging predictors; Breiman (2001) Random forests. *Machine Learning* 24 and 45.
- Freund and Schapire (1997) A decision-theoretic generalization of on-line learning (AdaBoost). *J. Computer and System Sciences* 55.
- Friedman (2001) Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29(5); Friedman (2002) Stochastic gradient boosting.
- Chen and Guestrin (2016) XGBoost; Ke et al. (2017) LightGBM; Prokhorenkova et al. (2018) CatBoost.
- Wolpert (1992) Stacked generalization; Geurts, Ernst and Wehenkel (2006) Extremely randomized trees.
- Strobl et al. (2007) Bias in random forest variable importance; Goldstein et al. (2015) ICE plots; Lundberg and Lee (2017) SHAP.
- Mitchell et al. (2019) Model cards for model reporting; Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, chapters 7, 10, 15, 16.
- scikit-learn User Guide: ensembles, permutation importance, partial dependence, pipelines and composite estimators, model persistence.